

Introduction to Machine Learning Methods

Neural Networks

Haris Gačanin

RWTH Aachen University

Outline

- Neural neuron models
- Multi-class classification

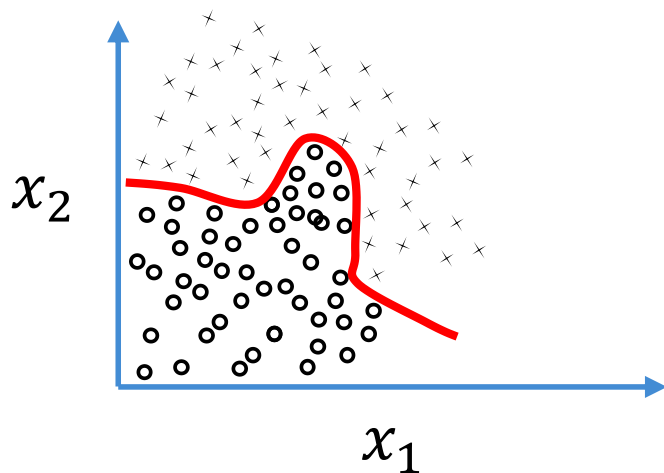
Reference: Chapter 18.7

Recap: Non-linear hypothesis

- By using logistic regression:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_2 x_1 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

- For regression models with large polynomials, we end up with large number of features in $g(\cdot)$



Recap: Non-linear hypothesis

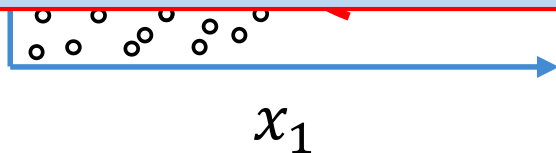
- By using logistic regression:

$$h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_2 x_1 + \theta_4 x_1^2 x_2 + \theta_5 x_1^3 x_2 + \theta_6 x_1 x_2^2 + \dots)$$

- For regression models with large polynomials, we end up with large number of features in $g(\cdot)$



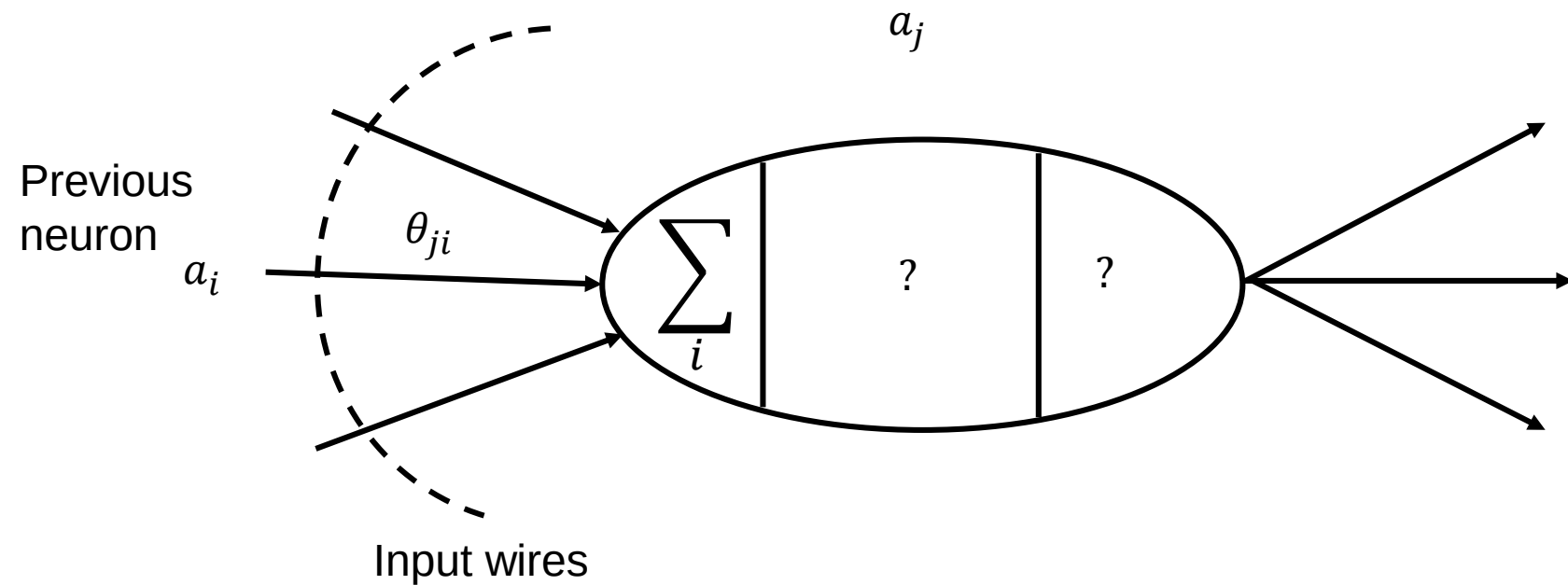
Logistic regression with many quadratic terms is not good approach to learn non-linear (i.e., complex) hypothesis.



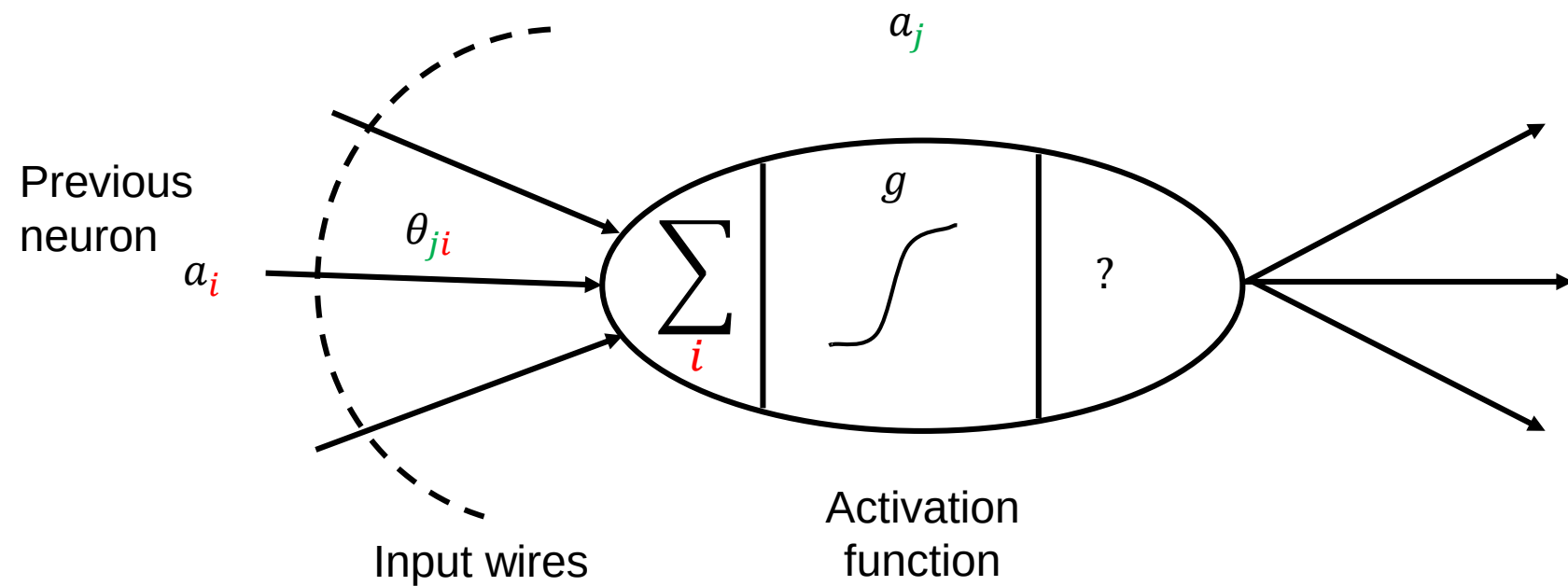


Mathematical model of a neuron

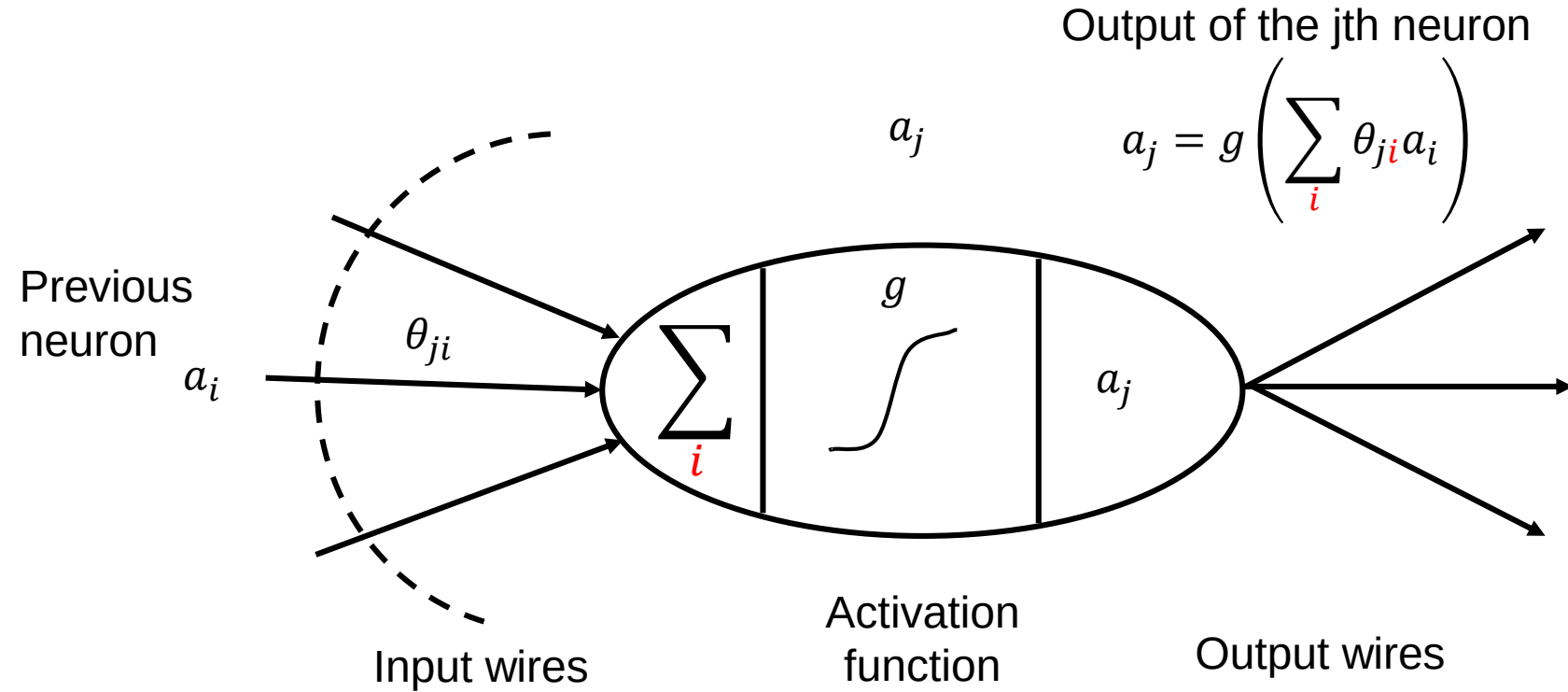
A neuron model a_j



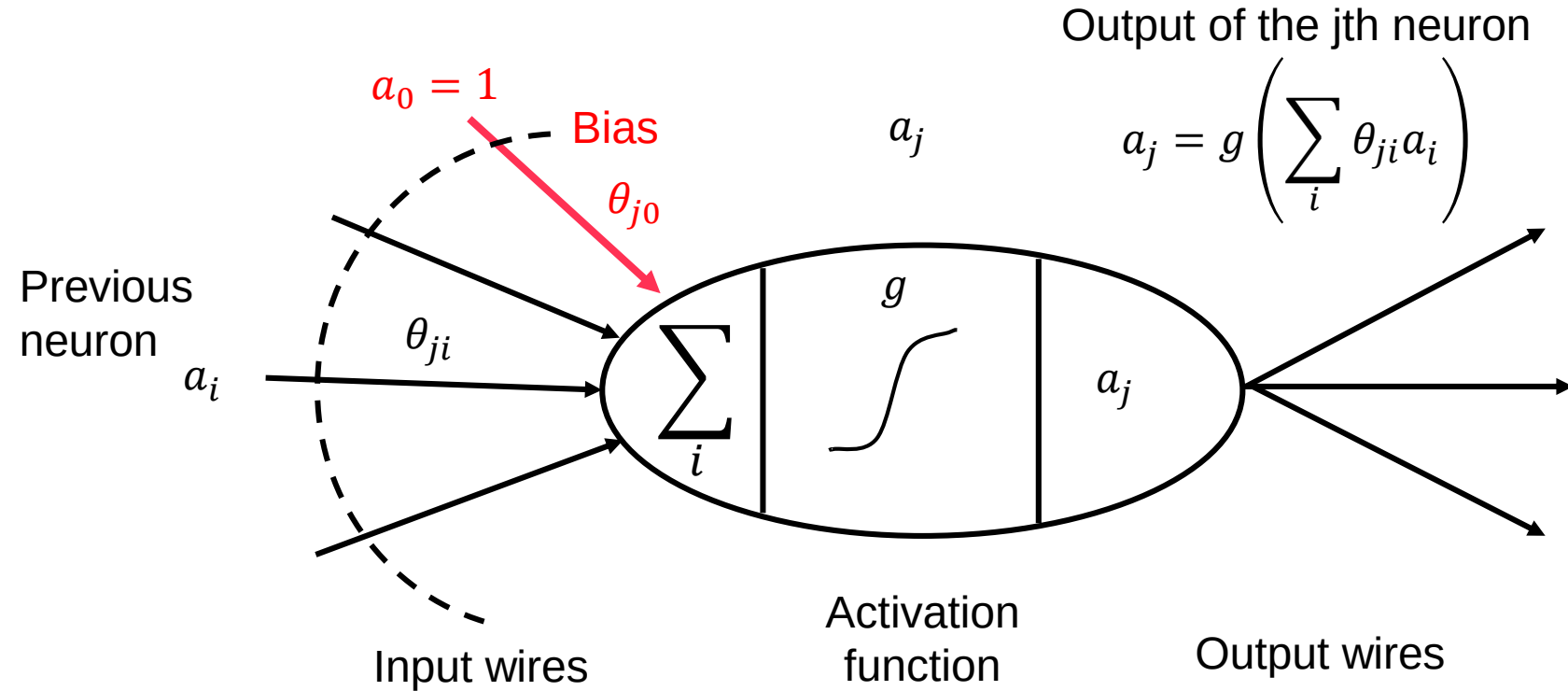
A neuron model a_j



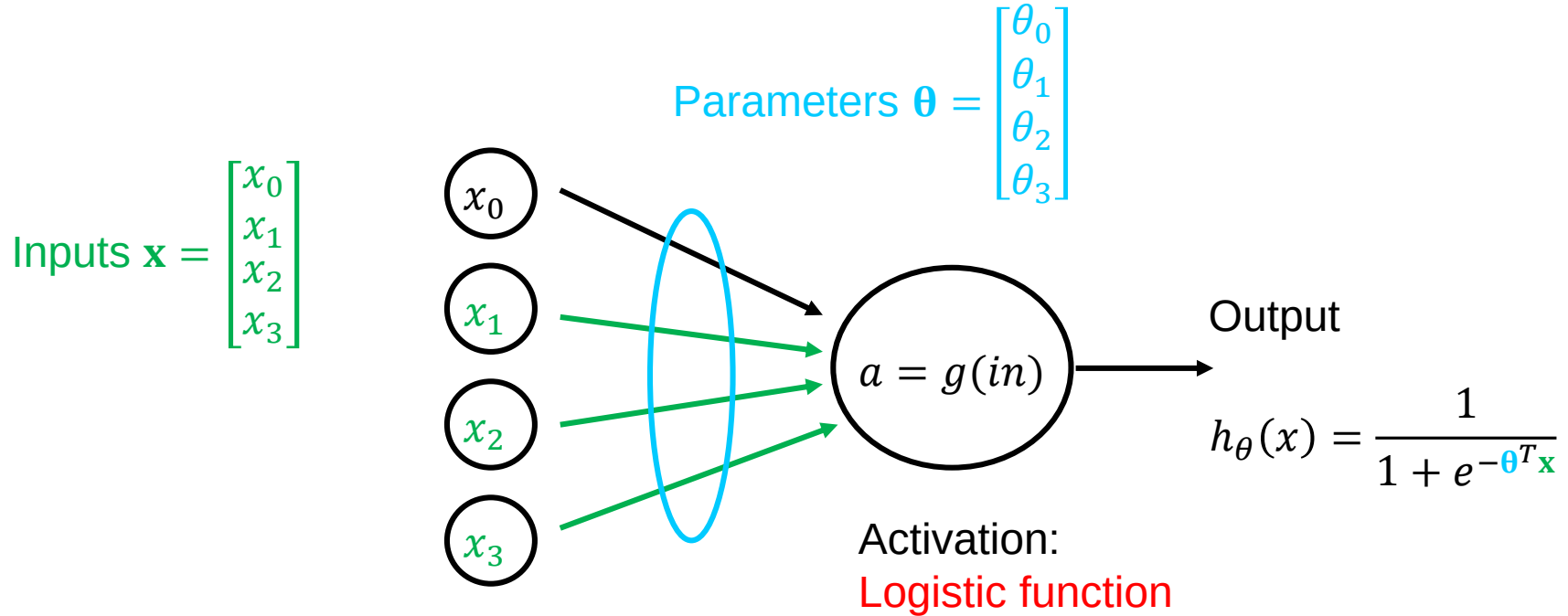
A neuron model a_j



A neuron model a_j



Definition of Neuron model with Logistic function

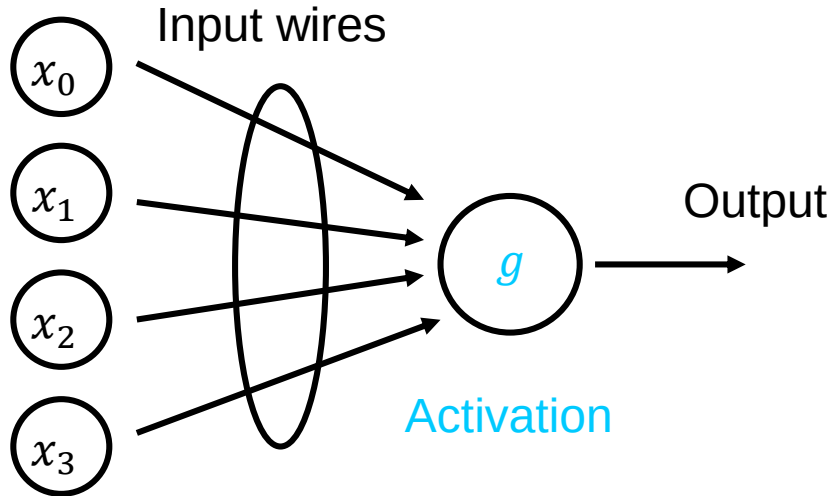




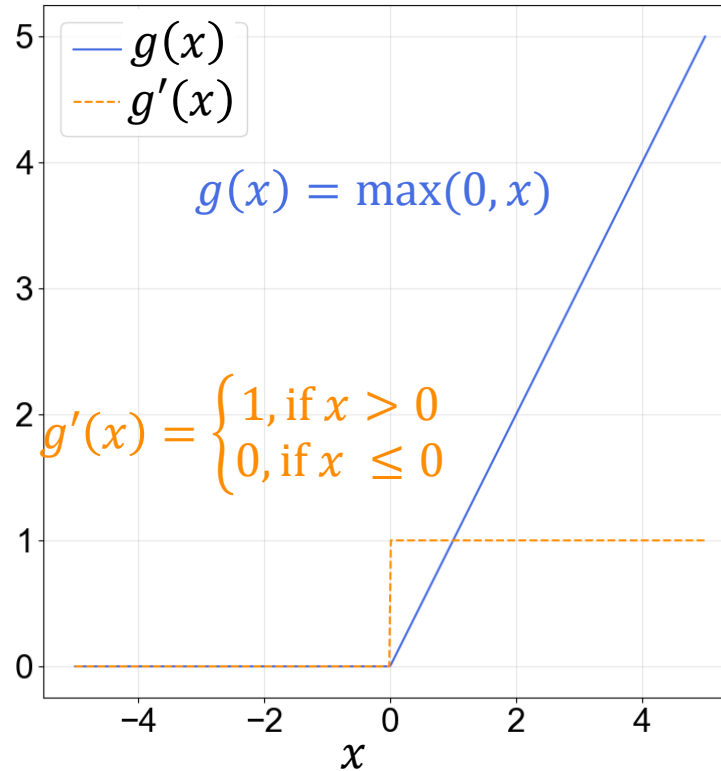
Activation functions

Activation function

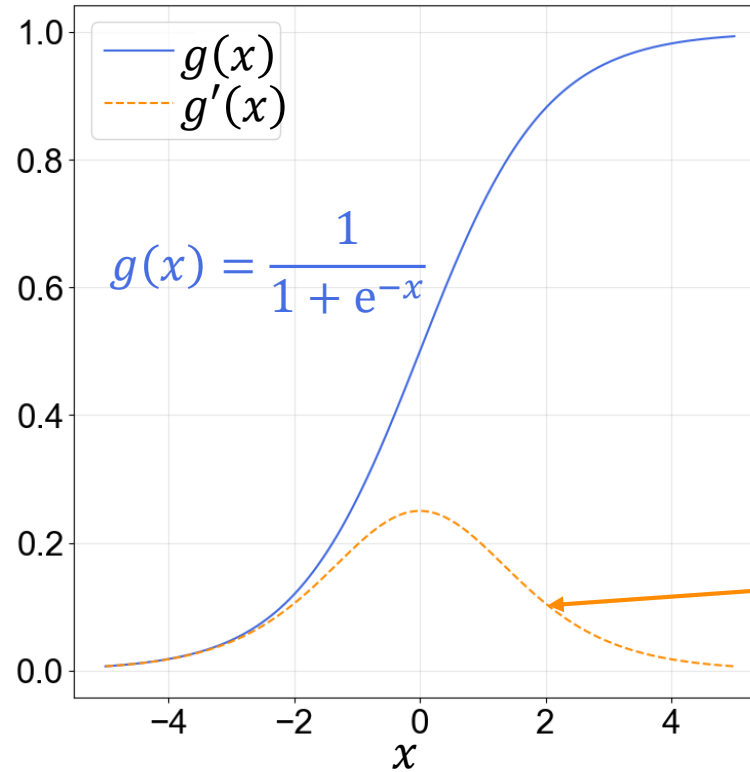
- Introduces a non-linearity into the network
- Function types
 - Linear
 - Logistic
 - Softmax
 - Tanh



Rectified linear unit (ReLU) activation function

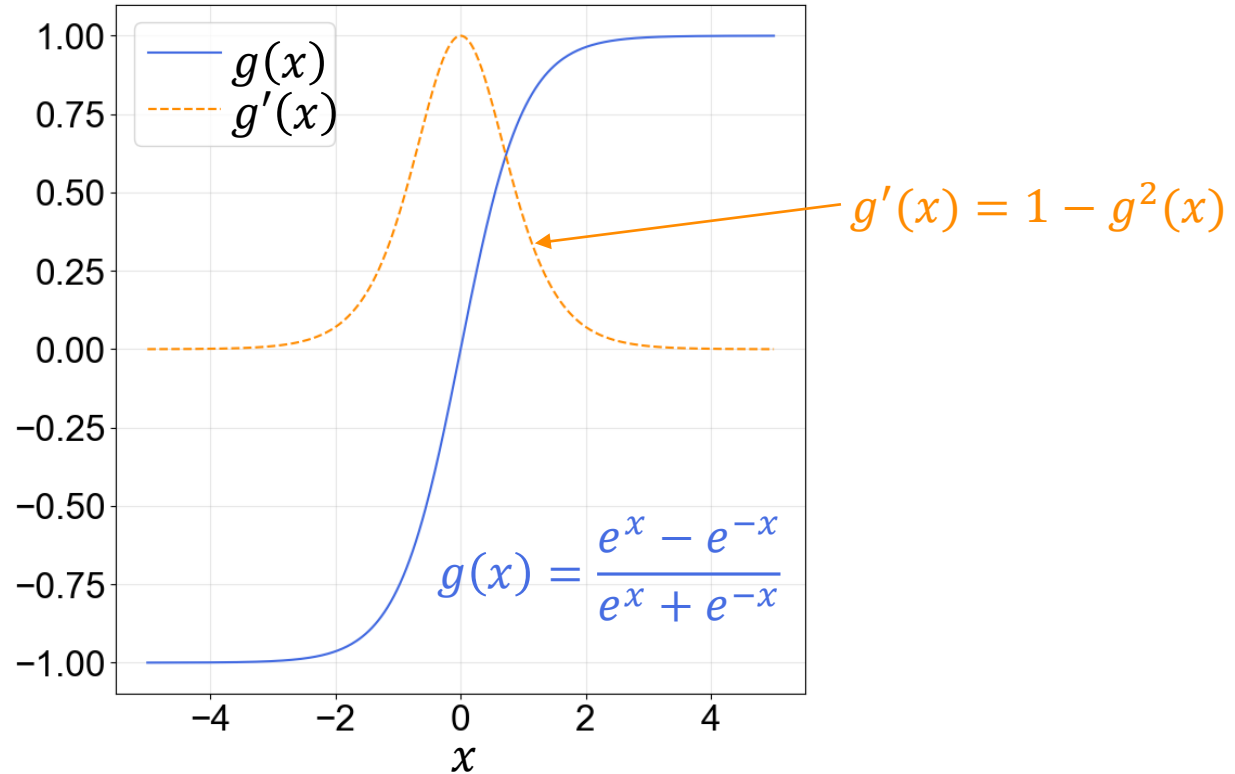


Logistic (Sigmoid) activation function



$$\begin{aligned}\frac{\partial g(x)}{\partial x} &= g'(x) \\ &= g(x)(1 - g(x))\end{aligned}$$

Tanh activation function

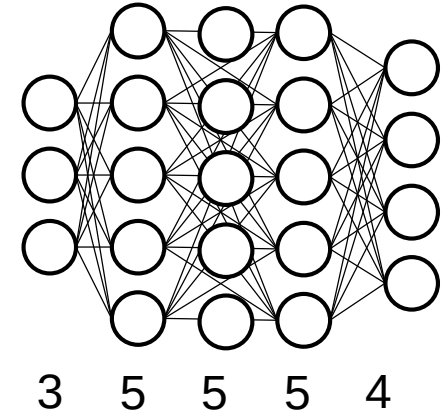
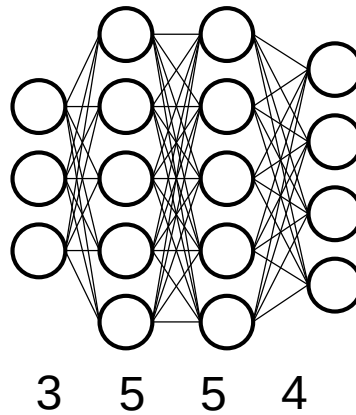
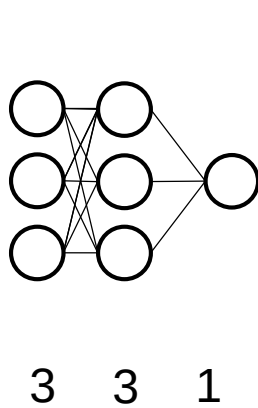




Neural networks

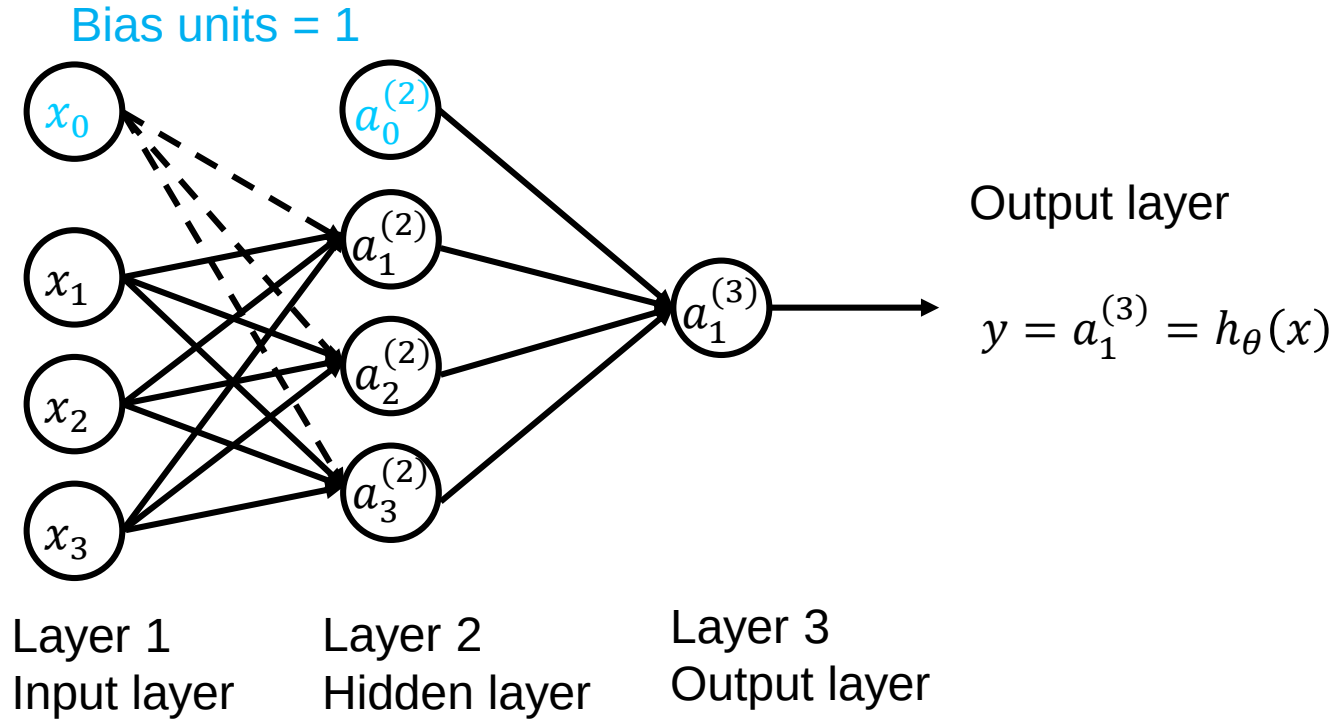
Example of Network Network architecture

- Bias units not included



Example (3, 3, 1)

$a_i^{(j)}$ = activation of unit i in layer j



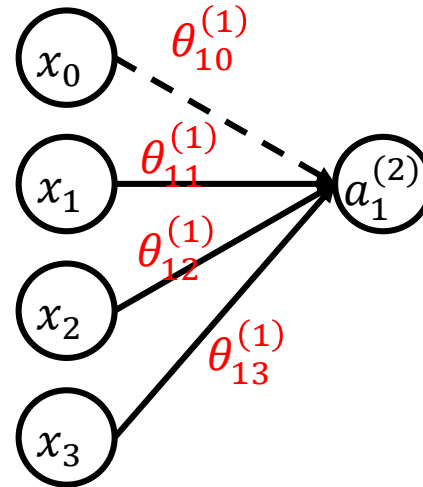
Hidden layer structure

$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

$$\boldsymbol{\theta}^{(1)} \in \mathbb{R}^{3 \times 4}$$

4 inputs

3 hidden units



Hidden layer structure

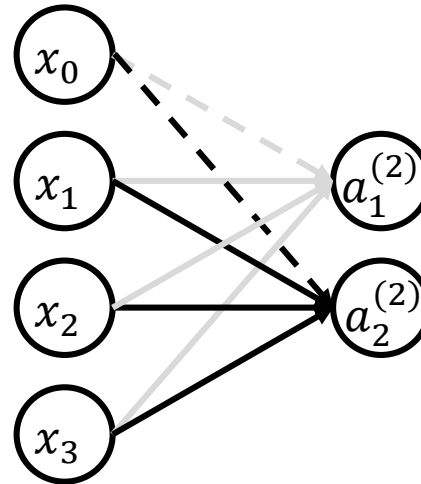
$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3)$$

$$\boldsymbol{\theta}^{(1)} \in \mathbb{R}^{3 \times 4}$$

4 inputs

3 hidden units



Hidden layer structure

$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

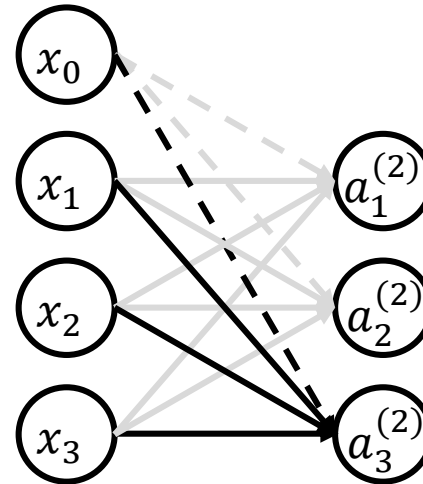
$$a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3)$$

$$\boldsymbol{\theta}^{(1)} \in \mathbb{R}^{3 \times 4}$$

4 inputs

3 hidden units



Hidden layer structure

$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3)$$

$$a_3^{(2)} = g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3)$$

$$\boldsymbol{\theta}^{(1)} \in \mathbb{R}^{3 \times 4}$$

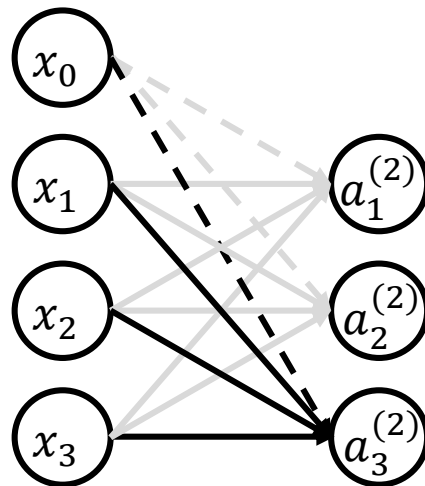
4 inputs

3 hidden units

Remark:

If network has s_j units in layer “ j ”, s_{j+1} units in layer “ $j + 1$ ”, then $\boldsymbol{\theta}^{(j)}$ will be of dimension

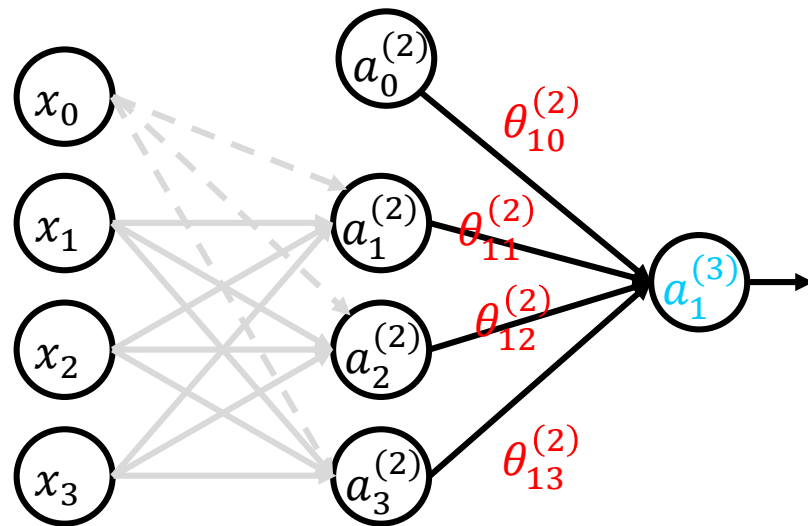
$$s_{j+1} \times s_j$$



Output layer

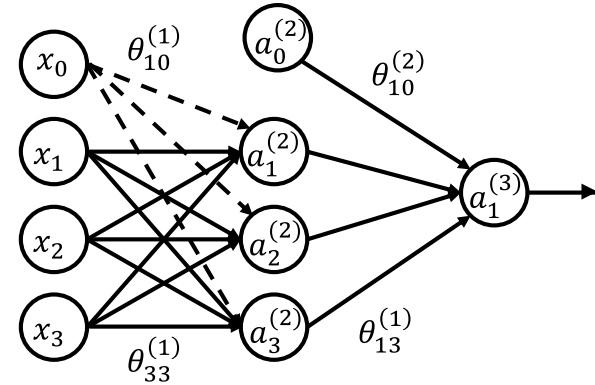
$$a_1^{(3)} = g\left(\boldsymbol{\theta}^{(2)T} \mathbf{a}^{(2)}\right) = g\left(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}\right)$$

$$\hat{y} = a_1^{(3)} = h_{\theta}(x)$$



Compact form of NN

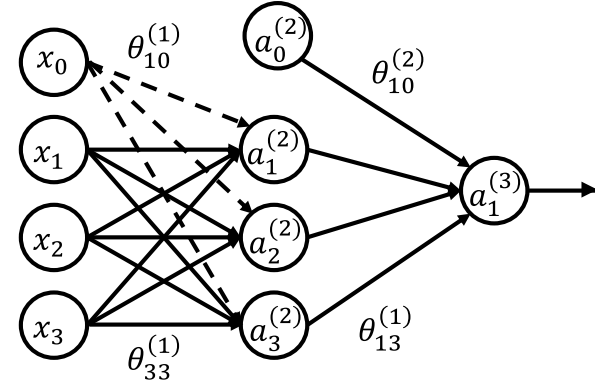
$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$



Compact form of NN

$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

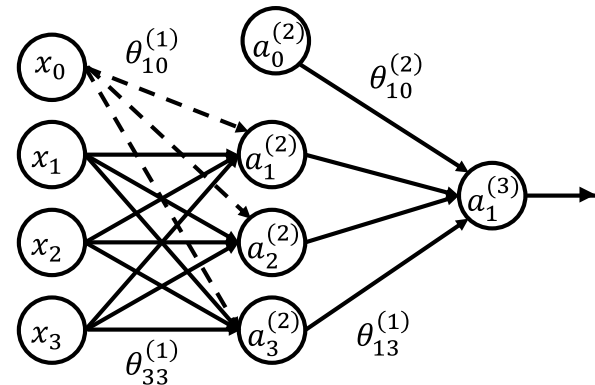


Compact form of NN

$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

$$a_3^{(2)} = g \left(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3 \right) = g \left(z_3^{(2)} \right)$$



Compact form of NN

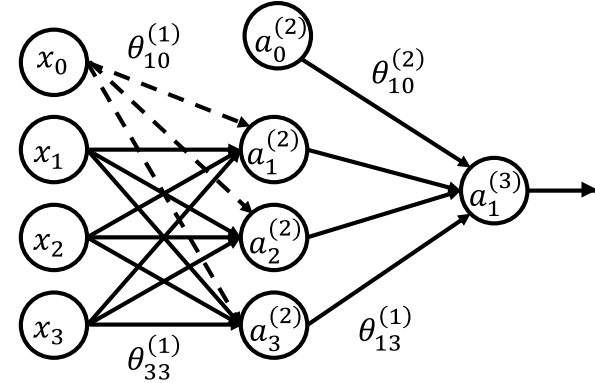
$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

$$a_3^{(2)} = g \left(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3 \right) = g \left(z_3^{(2)} \right)$$

- Generally, we have

$$z_k^{(2)} = \boldsymbol{\theta}^{(1)} \mathbf{x} = \theta_{k0}^{(1)} x_0 + \theta_{k1}^{(1)} x_1 + \theta_{k2}^{(1)} x_2 + \theta_{k3}^{(1)} x_3$$



Compact form of NN

$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

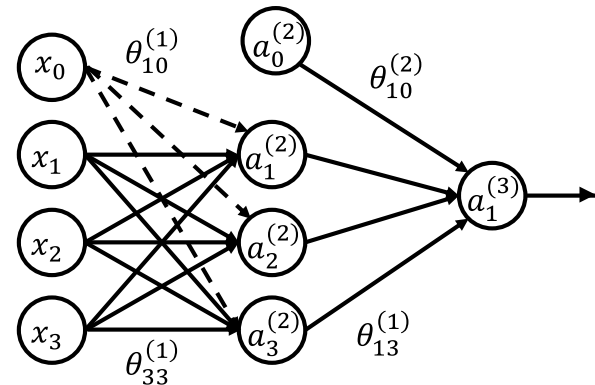
$$a_3^{(2)} = g \left(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3 \right) = g \left(z_3^{(2)} \right)$$

- Generally, we have

$$z_k^{(2)} = \boldsymbol{\theta}^{(1)} \mathbf{x} = \theta_{k0}^{(1)} x_0 + \theta_{k1}^{(1)} x_1 + \theta_{k2}^{(1)} x_2 + \theta_{k3}^{(1)} x_3$$

- Model output $h_{\theta}(x)$ is given by

$$\begin{aligned} a_1^{(3)} &= g \left(z_1^{(3)} \right) \\ &= g \left(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(1)} a_3^{(2)} \right) \end{aligned}$$

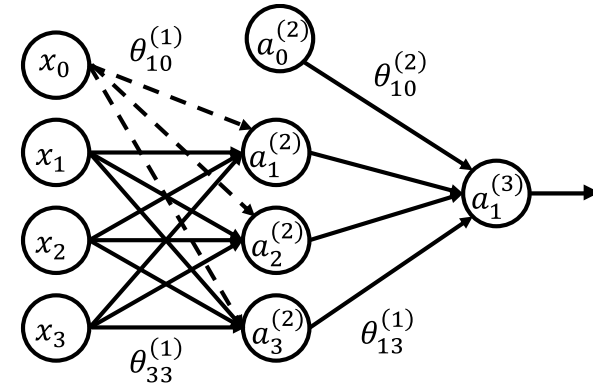




Basic principles of learning with NN

Learning principle in NNs

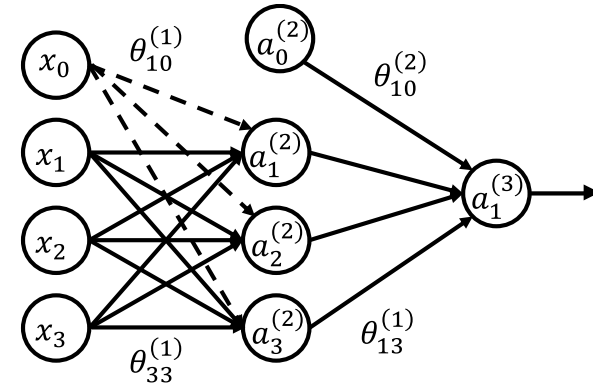
- NN output $h_{\theta}(x)$
 - $a_1^{(2)}, a_2^{(2)}, a_3^{(2)}$ are learnt as functions of input layer



Activation is similar to logistic regression

Learning principle in NNs

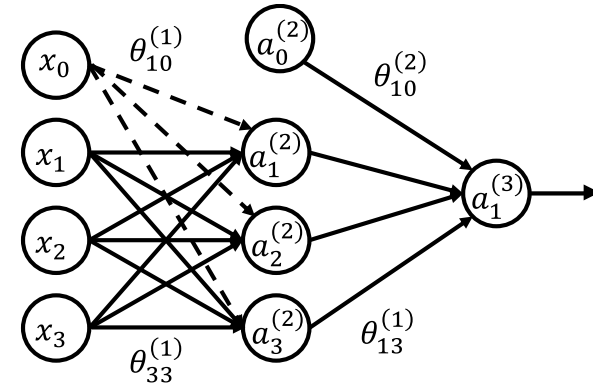
- NN output $h_{\theta}(x)$
 - $a_1^{(2)}, a_2^{(2)}, a_3^{(2)}$ are learnt as functions of input layer
 - Instead of features x_1, x_2, x_3 it feeds $a_1^{(2)}, a_2^{(2)}, a_3^{(2)}$ to “logistic regression”



Activation is similar to logistic regression

Learning principle in NNs

- NN output $h_{\theta}(x)$
 - $a_1^{(2)}, a_2^{(2)}, a_3^{(2)}$ are learnt as functions of input layer
 - Instead of features x_1, x_2, x_3 it feeds $a_1^{(2)}, a_2^{(2)}, a_3^{(2)}$ to “logistic regression”
- NN learns its own features
- **Forward propagation/computing** starts from **input layer** and **computes the following layer** units until we reach output



Activation is similar to logistic regression



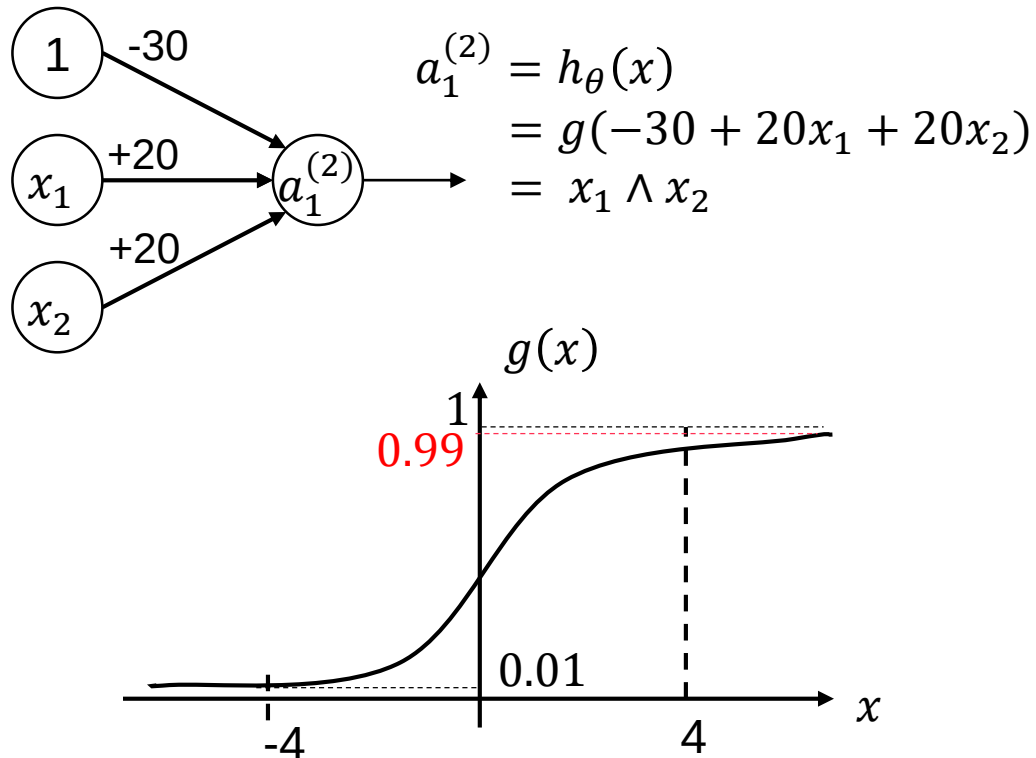
Intuitive examples

Boolean algebra

Simple example: AND

- $x_1, x_2 \in \{0,1\}$ $y = x_1 \wedge x_2$

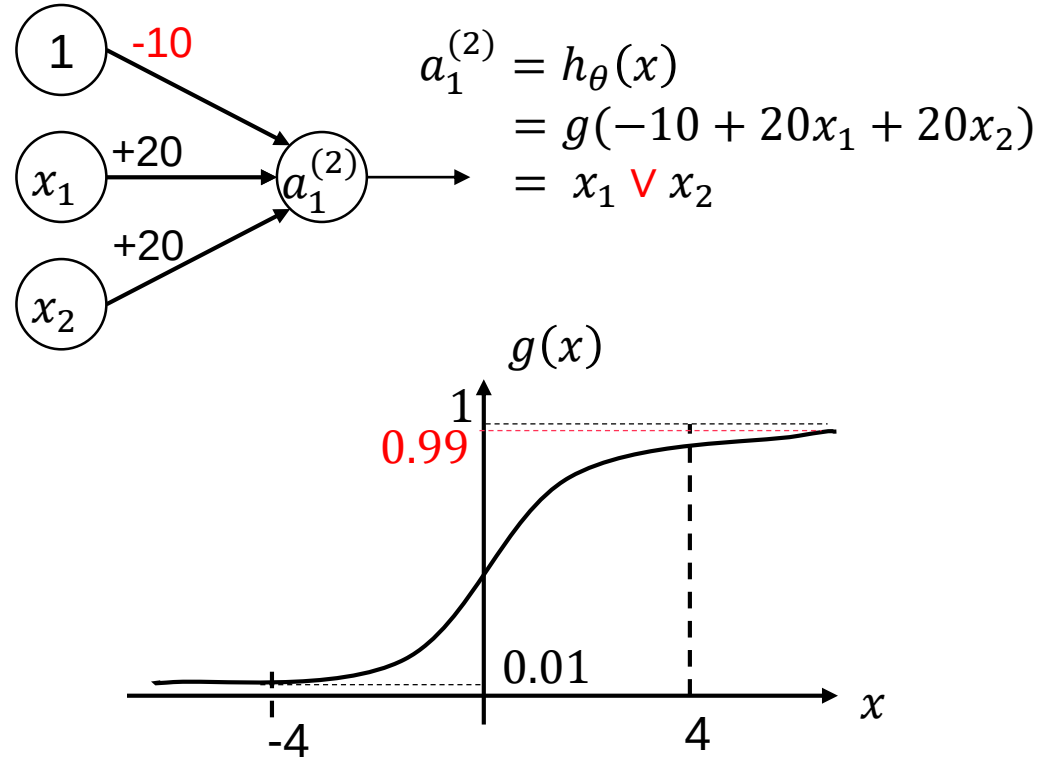
x_1	x_2	$h_\theta(x)$
0	0	$g(-30) \approx 0$
0	1	$g(-10) \approx 0$
1	0	$g(-10) \approx 0$
1	1	$g(10) \approx 1$



Simple example: OR

- $x_1, x_2 \in \{0,1\}$ $y = x_1 \vee x_2$

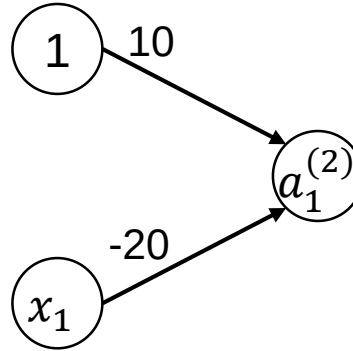
x_1	x_2	$h_\theta(x)$
0	0	$g(-10) \approx 0$
0	1	$g(10) \approx \mathbf{1}$
1	0	$g(10) \approx \mathbf{1}$
1	1	$g(\mathbf{30}) \approx 1$



Simple example: NOR

- $x_1 \in \{0,1\}$ $y = \neg x_1$

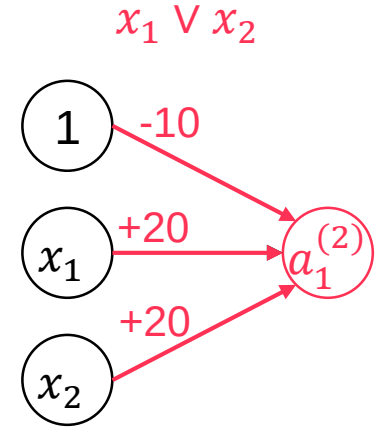
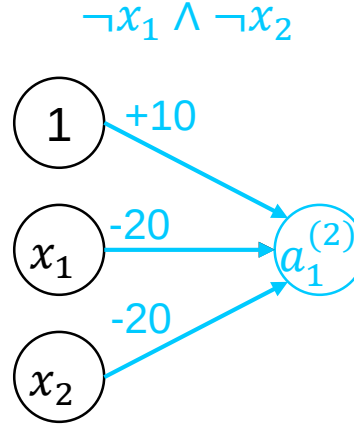
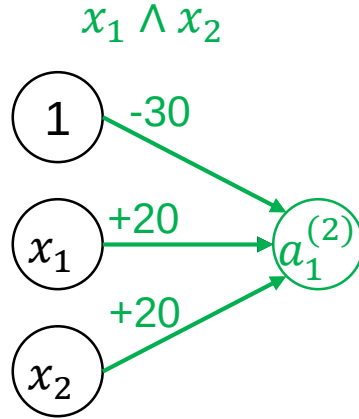
x_1	$h_\theta(x)$
0	$g(10) \approx 1$
1	$g(-10) \approx 0$



$$\begin{aligned} a_1^{(2)} &= h_\theta(x) \\ &= g(10 - 20x_1) \\ &= \neg x_1 \end{aligned}$$

How about combinations?

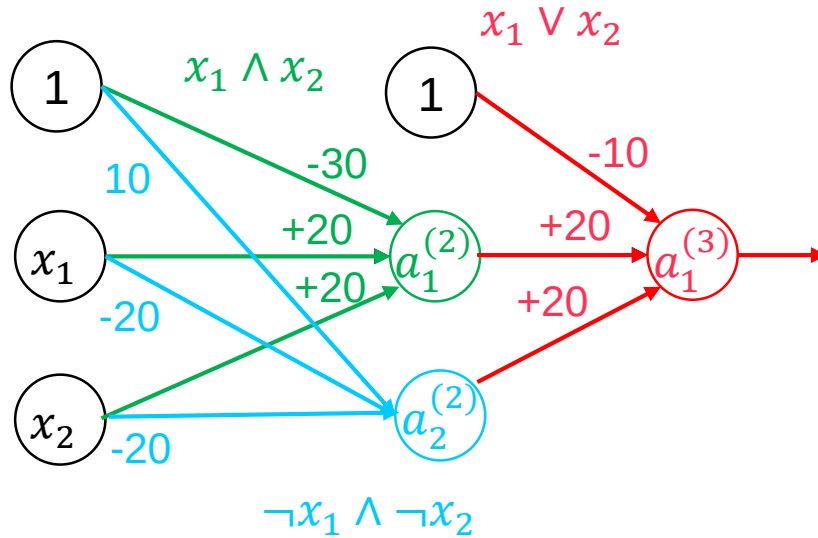
- $x_1, x_2 \in \{0,1\}$



Let's design $y = \text{XNOR}$ circuit...

XNOR circuit

- $x_1, x_2 \in \{0,1\}$



$$h_{\theta}(x) = a_1^{(3)} = g\left(-10 + 20a_1^{(2)} + 20a_2^{(2)}\right)$$

XNOR model output

$$\begin{aligned} a_1^{(3)} &= g\left(-10 + 20a_1^{(2)} + 20a_2^{(2)}\right) \\ &= g\left(-10 \cdot 1 + 20 \cdot g\left(z_1^{(2)}\right) + 20 \cdot g\left(z_2^{(2)}\right)\right) \\ &= g\left(-10 \cdot 1 + 20 \cdot g(-30 \cdot 1 + 20 \cdot x_1 + 20 \cdot x_2) + 20 \cdot g(10 \cdot 1 - 20 \cdot x_1 - 20 \cdot x_2)\right) \end{aligned}$$

XNOR model output

$$\begin{aligned}a_1^{(2)} &= g\left(-10 + 20a_1^{(2)} + 20a_1^{(2)}\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g\left(z_1^{(2)}\right) + 20 \cdot g\left(z_2^{(2)}\right)\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g(-30 \cdot 1 + 20 \cdot x_1 + 20 \cdot x_2) + 20 \cdot g(10 \cdot 1 - 20 \cdot x_1 - 20 \cdot x_2)\right)\end{aligned}$$

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\theta(x)$
0	0	0	1	$g(c) \approx 1$

XNOR model output

$$\begin{aligned}a_1^{(2)} &= g\left(-10 + 20a_1^{(2)} + 20a_1^{(2)}\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g\left(z_1^{(2)}\right) + 20 \cdot g\left(z_2^{(2)}\right)\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g(-30 \cdot 1 + 20 \cdot x_1 + 20 \cdot x_2) + 20 \cdot g(10 \cdot 1 - 20 \cdot x_1 - 20 \cdot x_2)\right)\end{aligned}$$

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\theta(x)$
0	0	0	1	$g(c) \approx 1$
0	1	0	0	$g(c) \approx 0$

XNOR model output

$$\begin{aligned}a_1^{(2)} &= g\left(-10 + 20a_1^{(2)} + 20a_1^{(2)}\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g\left(z_1^{(2)}\right) + 20 \cdot g\left(z_2^{(2)}\right)\right) \\&= g\left(-10 \cdot 1 + 20 \cdot g(-30 \cdot 1 + 20 \cdot x_1 + 20 \cdot x_2) + 20 \cdot g(10 \cdot 1 - 20 \cdot x_1 - 20 \cdot x_2)\right)\end{aligned}$$

x_1	x_2	$a_1^{(2)}$	$a_2^{(2)}$	$h_\theta(x)$
0	0	0	1	$g(c) \approx 1$
0	1	0	0	$g(c) \approx 0$
1	0	0	0	$g(c) \approx 0$
1	1	1	0	$g(c) \approx 1$



Multi-class classification

Binary vs. Multi-class classification problem

- Binary classification

$$y \in R^1$$

$$y = \{0 \text{ OR } 1\}$$

$$h_{\theta}(x) \in R^1$$

One output unit

Binary vs. Multi-class classification problem

- Binary classification

$$y \in R^1$$

$$y = \{0 \text{ OR } 1\}$$

$$h_{\theta}(x) \in R^1$$

One output unit

- K -class classification

$$\mathbf{y} = [y_1 y_2 \dots y_K]^T \in R^K$$

$$\text{e.g. } y_k = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \text{ OR } \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \end{bmatrix} \text{ OR } \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \text{ OR } \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{bmatrix}$$

$$(h_{\theta}(x))_k \in R^K$$

K output units

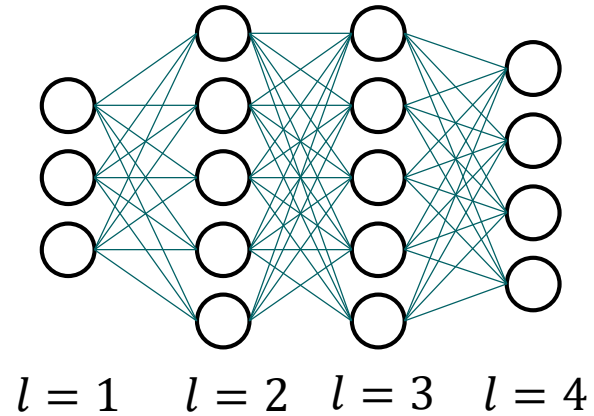
Example of a a four-class problem

- 4 class problem

L : total number of hidden layers l
($L = 4$)

Input

Output

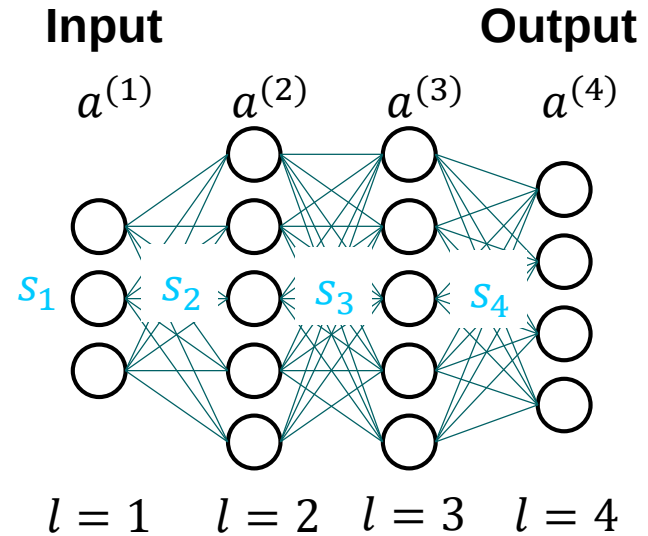


Example of a a four-class problem

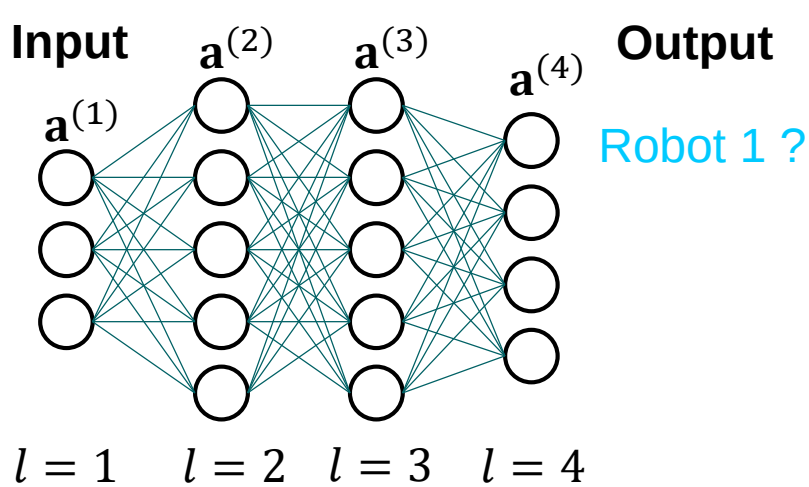
- 4 class problem

L : total number of hidden layers l
($L = 4$)

s_l : number of units (not counting bias unit)
($s_1 = 3, s_2 = s_3 = 5, s_4 = 4$)



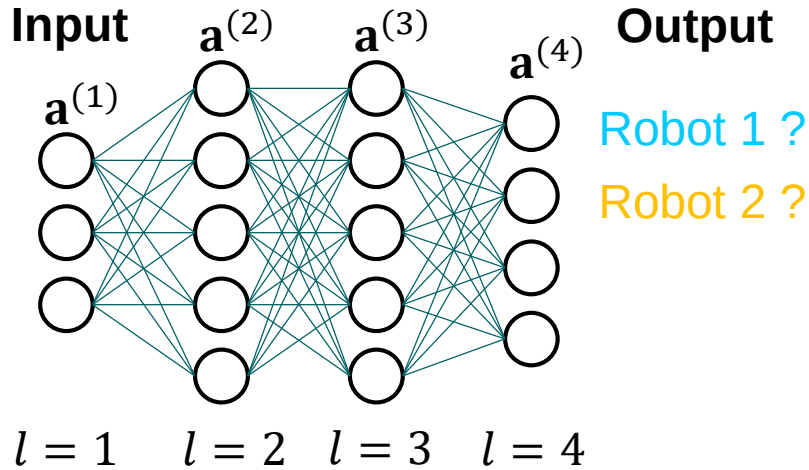
Example: Structure for multi-class classification



$$(h_{\theta}(x))_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

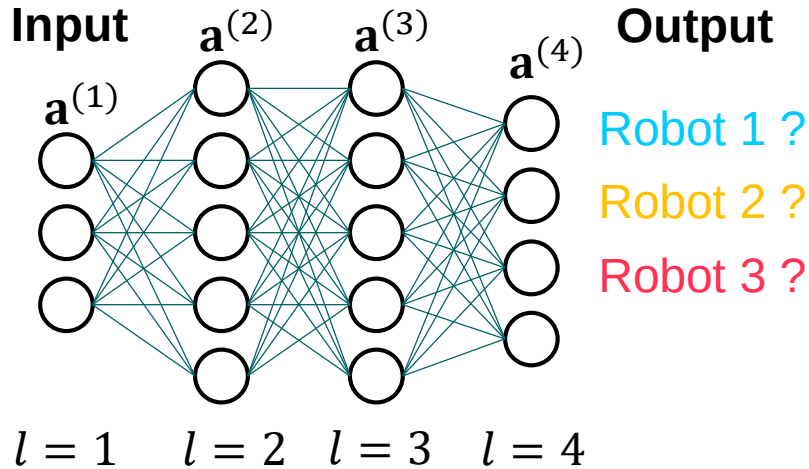
Robot 1

Example: Structure for multi-class classification



$$(h_{\theta}(x))_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 1}$$
$$(h_{\theta}(x))_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 2}$$

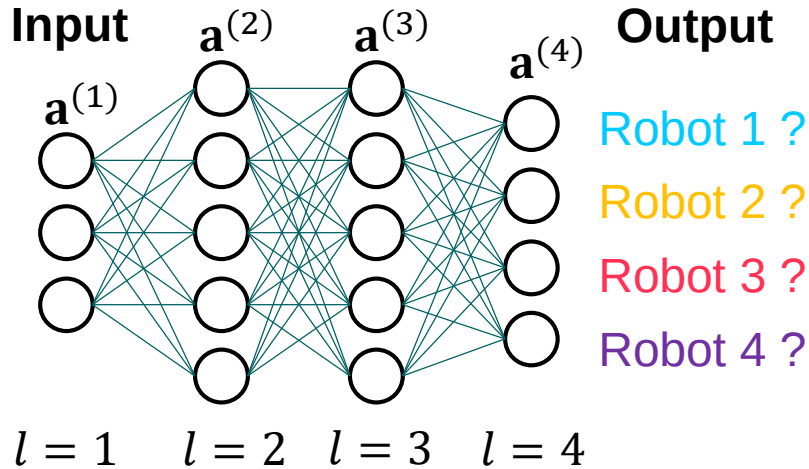
Example: Structure for multi-class classification



$$(h_{\theta}(x))_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 1}$$
$$(h_{\theta}(x))_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 2}$$

$$(h_{\theta}(x))_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{Robot 3}$$

Example: Structure for multi-class classification



$$(h_{\theta}(x))_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 1}$$

$$(h_{\theta}(x))_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad \text{Robot 2}$$

$$(h_{\theta}(x))_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \quad \text{Robot 3}$$

$$(h_{\theta}(x))_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \quad \text{Robot 4}$$

Training process

- Training set: $(x^{(i)}, y^{(i)})_{i=0 \sim m}$
- For each training set, the output $y^{(i)}$ is one of the four classification targets:

$$y_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \quad y_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad y_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \quad y_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix}$$

Robot 1

Robot 2

Robot 3

Robot 4

Goal is to have

$$\left(h_{\theta}(x^{(i)}) \right)_k \approx y_k^{(i)} \in R^K$$



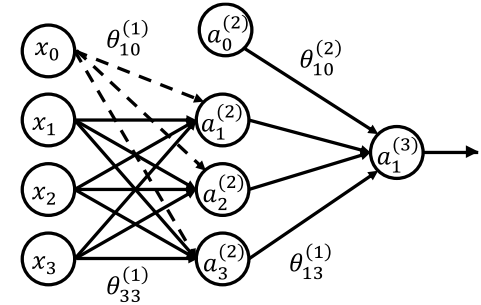
How to compute the parameters

Backpropagation algorithm

Recall the relationships

- Recal the relationships:

$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$



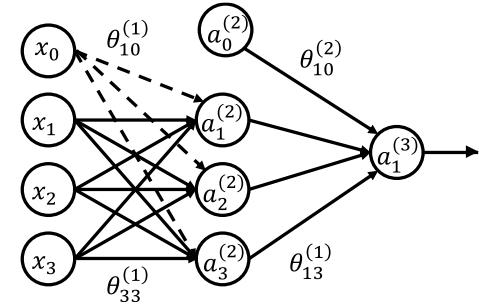
Recall the relationships

- Recal the relationships:

$$a_1^{(2)} = g \left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3 \right) = g \left(z_1^{(2)} \right)$$

$$a_2^{(2)} = g \left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3 \right) = g \left(z_2^{(2)} \right)$$

$$a_3^{(2)} = g \left(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3 \right) = g \left(z_3^{(2)} \right)$$



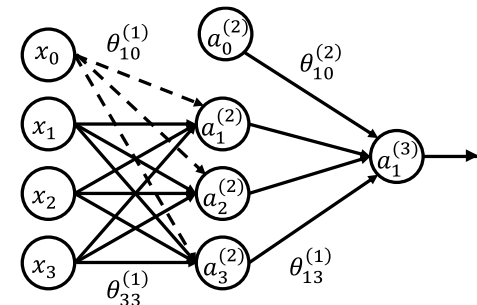
Recall the relationships

- Recal the relationships:

$$a_1^{(2)} = g\left(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3\right) = g\left(z_1^{(2)}\right)$$

$$a_2^{(2)} = g\left(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3\right) = g\left(z_2^{(2)}\right)$$

$$a_3^{(2)} = g\left(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3\right) = g\left(z_3^{(2)}\right)$$



- Vectorized parameters are:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \mathbf{z}^{(2)} = \begin{bmatrix} z_1^{(2)} \\ z_2^{(2)} \\ z_3^{(2)} \end{bmatrix} \quad \mathbf{a}^{(2)} = \begin{bmatrix} a_0^{(2)} \\ a_1^{(2)} \\ a_2^{(2)} \\ a_3^{(2)} \end{bmatrix}$$

Backpropagation algorithm

- **Goal** is to compute gradient $\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}}$ of the cost function $J(\boldsymbol{\theta})$

Backpropagation algorithm

- **Goal** is to compute gradient $\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}}$ of the cost function $J(\boldsymbol{\theta})$
- **Forward computation**
 - Feed the inputs through the network and make predictions per layer
 - Compute a cost function (i.e., error with respect to the true label) at the output
- **Backward computation**
 - Propagate the gradients of the error from the output layer to the input layer

Backpropagation algorithm

- **Goal** is to compute gradient $\frac{\partial J(\theta)}{\partial \theta^{(l)}}$ of the cost function $J(\theta)$
- **Forward computation**
 - Feed the inputs through the network and make predictions per layer
 - Compute a cost function (i.e., error with respect to the true label) at the output
- **Backward computation**
 - Propagate the gradients of the error from the output layer to the input layer
- **Parameter update with gradient descent computation**
 - We update the parameters (i.e., connections between neurons) by taking a step in the opposite direction of the error gradients $\frac{\partial J(\theta)}{\partial \theta_{ij}^{(l)}}$

Backpropagation algorithm

- **Goal** is to compute gradient $\frac{\partial J(\theta)}{\partial \theta^{(l)}}$ of the cost function $J(\theta)$
- **Forward computation**
 - Feed the inputs through the network and make predictions per layer
 - Compute a cost function (i.e., error with respect to the true label) at the output
- **Backward computation**
 - Propagate the gradients of the error from the output layer to the input layer
- **Parameter update with gradient descent computation**
 - We update the parameters (i.e., connections between neurons) by taking a step in the opposite direction of the error gradients $\frac{\partial J(\theta)}{\partial \theta_{ij}^{(l)}}$
- $J(\theta) \rightarrow$ **Let's recall once again!**

Recall the $J(\theta)$ for logistic regression with L_2 regularization

- Recall the cost of the logistic regression function, given by:

$$J_{LG}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{\theta}(x^{(i)})_k \right) + \frac{\lambda}{2m} \sum_{k=1}^K \sum_{j=1}^n (\theta_{kj})^2$$

Note: bias term θ_0 is NOT regularized

Note: it is the same expression!

Recall the $J(\theta)$ for logistic regression with L_2 regularization

- Recall the cost of the logistic regression function, given by:

$$J_{LG}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{\theta}(x^{(i)})_k \right) + \frac{\lambda}{2m} \sum_{k=1}^K \sum_{j=1}^n (\theta_{kj})^2$$

$$J_{NN}(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{\theta}(x^{(i)})_k \right) + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{j=1}^{s_{l+1}} \sum_{i=1}^{s_l} (\theta_{ji}^{(l)})^2$$

Note: bias term θ_0 is NOT regularized

Note: it is the same expression!

Compact form of $J(\theta)$ for neural network with L_2 regularization

- The regularized cost function for binary case generalizes logistic regression by summing the loss over all output units and adding penalties for all parameters

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K \left[y_k^{(i)} \log(h_{\theta}(x^{(i)})_k) + (1 - y_k^{(i)}) \log(1 - h_{\theta}(x^{(i)})_k) \right] \\ + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{j=1}^{s_{l+1}} \sum_{i=1}^{s_l} (\theta_{ji}^{(l)})^2$$

K output units (class)

$h_{\theta}(x^{(i)})_k$ predicted probability of class k

$y_k^{(i)} \in \{0,1\}$ output

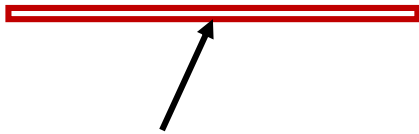
- **Note:** The regularization term is sum of squared weights across all layers

Special case: Cost function $J(\theta)$ for NN with softmax output

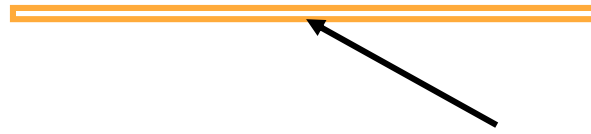
- For a multiclass classification with softmax activation:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \log \left(h_{\theta}(x^{(i)})_k \right) + \frac{\lambda}{2m} \sum_{l=1}^{L-1} \sum_{j=1}^{s_{l+1}} \sum_{i=1}^{s_l} \left(\theta_{ji}^{(l)} \right)^2$$

= Cross-entropy loss + λ · Weight Decay (smoothing)



This term defines how well the network fits the data



This term penalizes large weights and reduces overfitting (i.e., smoothing)

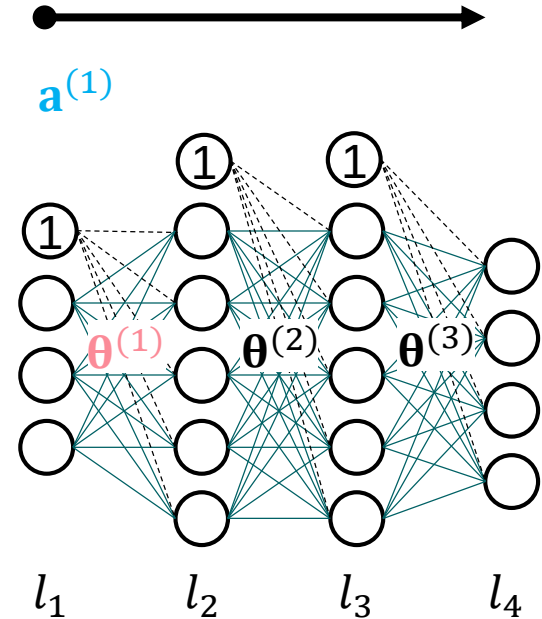


Forward computation

Forward computation

- Given one training example (x, y) :

$$\begin{aligned} \mathbf{a}^{(1)} &= \mathbf{x} \in R^{3 \times 1} && \text{(add bias } a_0^{(1)}) \\ \mathbf{z}^{(2)} &= \boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)} && \boldsymbol{\theta}^{(1)} \in R^{5 \times 3} \end{aligned}$$

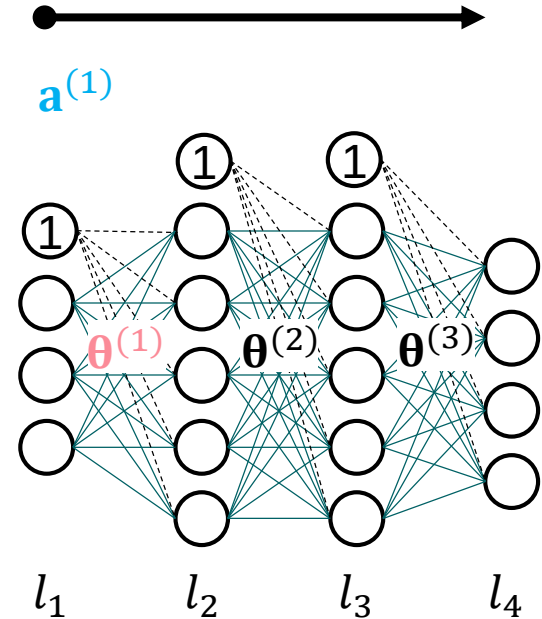


Forward computation

- Given one training example (x, y) :

$$\begin{aligned} \mathbf{a}^{(1)} &= \mathbf{x} \in R^{3 \times 1} && (\text{add bias } a_0^{(1)}) \\ \mathbf{z}^{(2)} &= \boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)} && \boldsymbol{\theta}^{(1)} \in R^{5 \times 3} \end{aligned}$$

Note: The bias is a parameter with a fixed value during that forward pass.



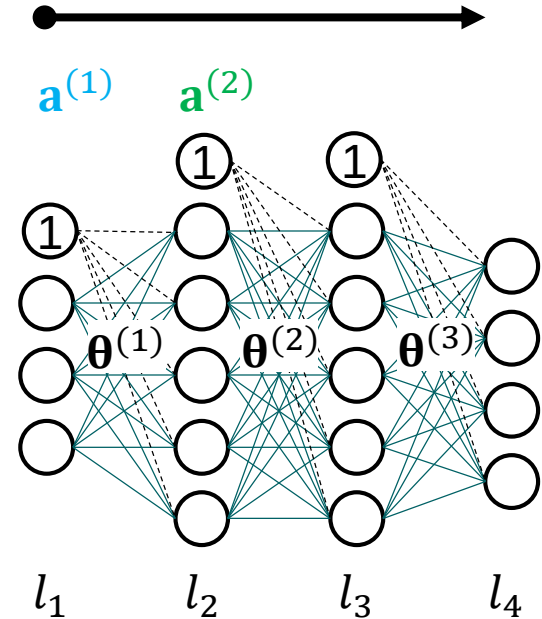
Forward computation

- Given one training example (x, y) :

$$\mathbf{a}^{(1)} = \mathbf{x} \in R^{3 \times 1} \quad (\text{add bias } a_0^{(1)})$$

$$\mathbf{z}^{(2)} = \boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)} \quad \boldsymbol{\theta}^{(1)} \in R^{5 \times 3}$$

$$\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)}) \in R^{5 \times 1} \quad (\text{add bias } a_0^{(2)})$$



Forward computation

- Given one training example (x, y) :

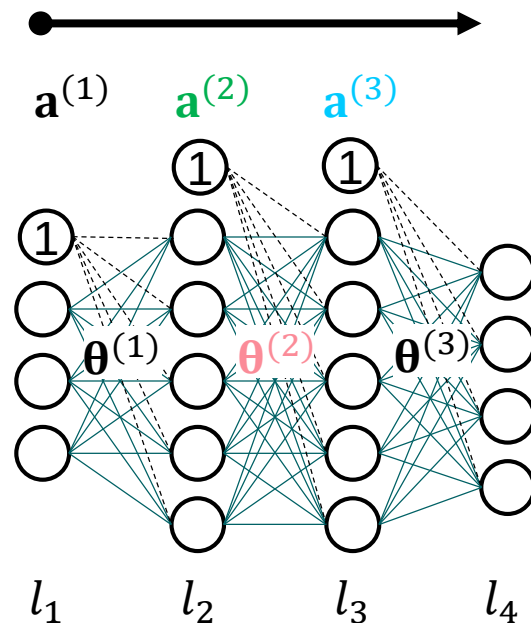
$$\mathbf{a}^{(1)} = \mathbf{x} \in R^{3 \times 1} \quad (\text{add bias } a_0^{(1)})$$

$$\mathbf{z}^{(2)} = \boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)} \quad \boldsymbol{\theta}^{(1)} \in R^{5 \times 3}$$

$$\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)}) \in R^{5 \times 1} \quad (\text{add bias } a_0^{(2)})$$

$$\mathbf{z}^{(3)} = \boldsymbol{\theta}^{(2)} \mathbf{a}^{(2)} \quad \boldsymbol{\theta}^{(2)} \in R^{5 \times 5}$$

$$\mathbf{a}^{(3)} = g(\mathbf{z}^{(3)}) \in R^{4 \times 1} \quad (\text{add bias } a_0^{(3)})$$



Forward computation

- Given one training example (x, y) :

$$\mathbf{a}^{(1)} = \mathbf{x} \in R^{3 \times 1} \quad (\text{add bias } a_0^{(1)})$$

$$\mathbf{z}^{(2)} = \boldsymbol{\theta}^{(1)} \mathbf{a}^{(1)} \quad \boldsymbol{\theta}^{(1)} \in R^{5 \times 3}$$

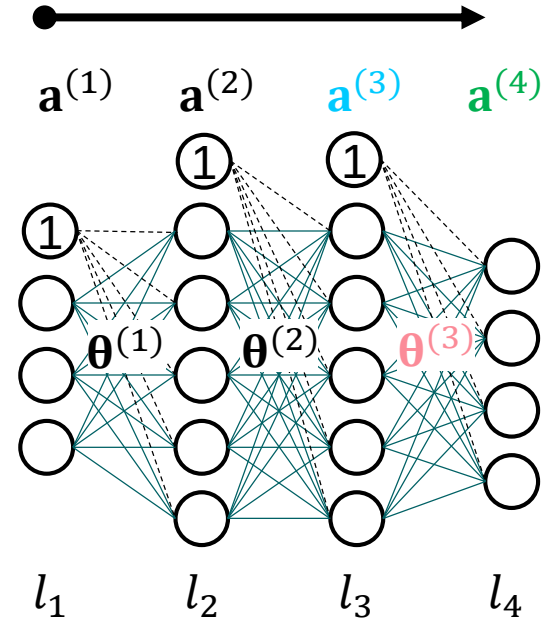
$$\mathbf{a}^{(2)} = g(\mathbf{z}^{(2)}) \in R^{5 \times 1} \quad (\text{add bias } a_0^{(2)})$$

$$\mathbf{z}^{(3)} = \boldsymbol{\theta}^{(2)} \mathbf{a}^{(2)} \quad \boldsymbol{\theta}^{(2)} \in R^{5 \times 5}$$

$$\mathbf{a}^{(3)} = g(\mathbf{z}^{(3)}) \in R^{4 \times 1} \quad (\text{add bias } a_0^{(3)})$$

$$\mathbf{z}^{(4)} = \boldsymbol{\theta}^{(3)} \mathbf{a}^{(3)} \quad \boldsymbol{\theta}^{(3)} \in R^{4 \times 5}$$

$$h_{\theta}(x) = \mathbf{a}^{(4)} = g(\mathbf{z}^{(4)}) \in R^{4 \times 1}$$





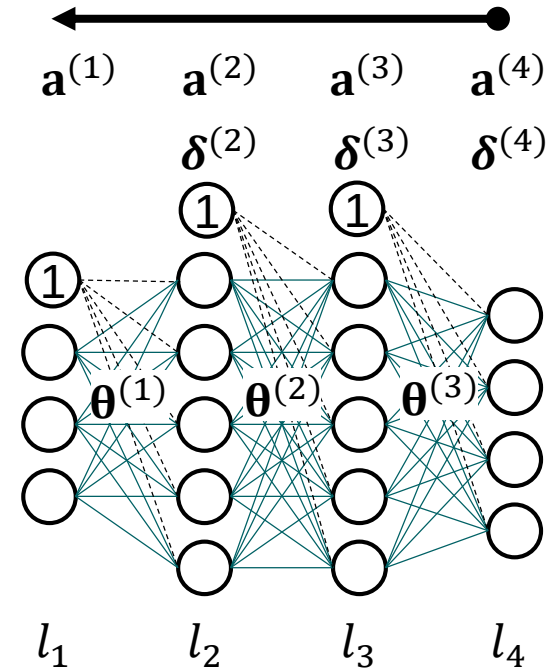
Backward computation

Definitions

- An error term of the layer l is given by

$$\delta^{(l)} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(l)}}$$

- Elements of $\delta^{(l)}$ given by $\left[\delta_j^{(l)}\right]_{s_j \times 1}$ define an error term of the j th node in layer l (i.e., $a_j^{(l)}$)
- Target output $\mathbf{y}_k \in R^4$ (i.e., $k = 1, 2, \dots, K$ **fixed targets** from our robotic example $K = 4$)
- Note:** s_j is number of units in the j th layer **not counting bias unit**



Output error layer computation with cross-entropy/regularization

- For each output unit l we compute error terms $\delta^{(l)}$
- For one training example $m = 1$, we have

$$J(\theta) = - \sum_{k=1}^4 \left[y_k \log \left(a_k^{(4)} \right) + (1 - y_k) \log \left(1 - a_k^{(4)} \right) \right] + \frac{\lambda}{2m} \sum_{l=1}^3 \sum_{i=1}^{s_{l+1}} \sum_{j=1}^{s_l} \left(\theta_{ij}^{(l)} \right)^2$$

- The output error is computed by

$$\delta_k^{(4)} = \frac{\partial J(\theta)}{\partial z_k^{(4)}} = \frac{\partial J_{\text{cross-entropy}}(\theta)}{\partial z_k^{(4)}} + \frac{\partial J_{\text{reg}}(\theta)}{\partial z_k^{(4)}}$$

Derivative of the regularization term $J_{reg}(\theta)$

- The regularization term depends only on weights:

$$J_{reg}(\theta) = \frac{\lambda}{2m} \sum_{l=1}^3 \sum_{i=1}^{s_{l+1}} \sum_{j=1}^{s_l} \left(\theta_{ij}^{(l)} \right)^2$$

- Since there is no dependency on z_k^L , a_k^L or $g(z_k^L)$, the derivative

$$\frac{\partial J_{reg}(\theta)}{\partial z_k^{(4)}} = 0$$

Output layer $l = 4$ error computation

- The output layer $l = 4$ the error is computed by

$$\delta_k^{(4)} = \frac{\partial J(\theta)}{\partial z_k^{(4)}} = \frac{\partial J_{\text{cross-entropy}}(\theta)}{\partial z_k^{(4)}} + \frac{\partial J_{\text{reg}}(\theta)}{\partial z_k^{(4)}} = \frac{\partial J_{\text{cross-entropy}}(\theta)}{\partial a_k^{(4)}} \frac{\partial a_k^{(4)}}{\partial z_k^{(4)}} + 0$$

Output layer $l = 4$ error computation

- The output layer $l = 4$ the error is computed by

$$\begin{aligned}\delta_k^{(4)} &= \frac{\partial J(\theta)}{\partial z_k^{(4)}} = \frac{\partial J_{\text{cross-entropy}}(\theta)}{\partial z_k^{(4)}} + \frac{\partial J_{\text{reg}}(\theta)}{\partial z_k^{(4)}} = \frac{\partial J_{\text{cross-entropy}}(\theta)}{\partial a_k^{(4)}} \frac{\partial a_k^{(4)}}{\partial z_k^{(4)}} + 0 \\ &= \left(-\frac{y_k}{a_k^{(4)}} + \frac{1 - y_k}{1 - a_k^{(4)}} \right) \underbrace{a_k^{(4)} (1 - a_k^{(4)})}_{\text{Derivative of a sigmoid function (see slides below)}} = -y_k (1 - a_k^{(4)}) + a_k^{(4)} (1 - y_k) \\ &= a_k^{(4)} - y_k\end{aligned}$$

Layer 4 (output):

$$\boldsymbol{\delta}^{(4)} = (\mathbf{a}^{(4)} - \mathbf{y}) \in R^{4 \times 1}$$

Computation of error terms $\delta^{(l)}$

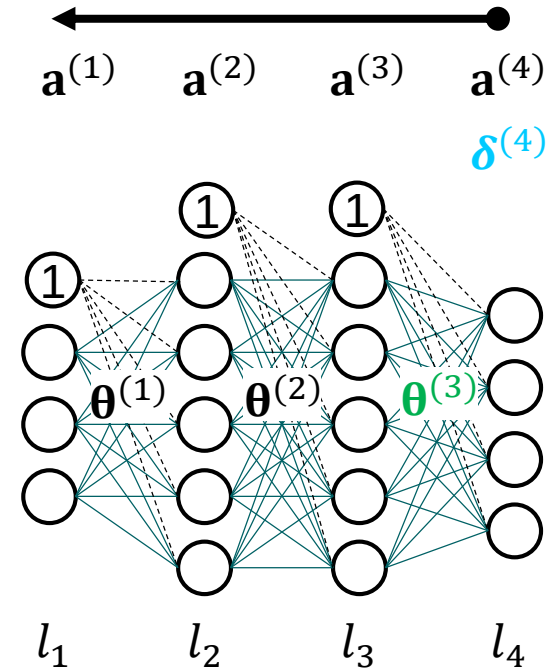
- The goal is to represent $\delta^{(l)} = f'(\delta^{(l+1)})$

- By definition

$$\delta^{(l)} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(l)}}$$

- Or equivalently

$$\delta_k^{(l)} = \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l)}}$$

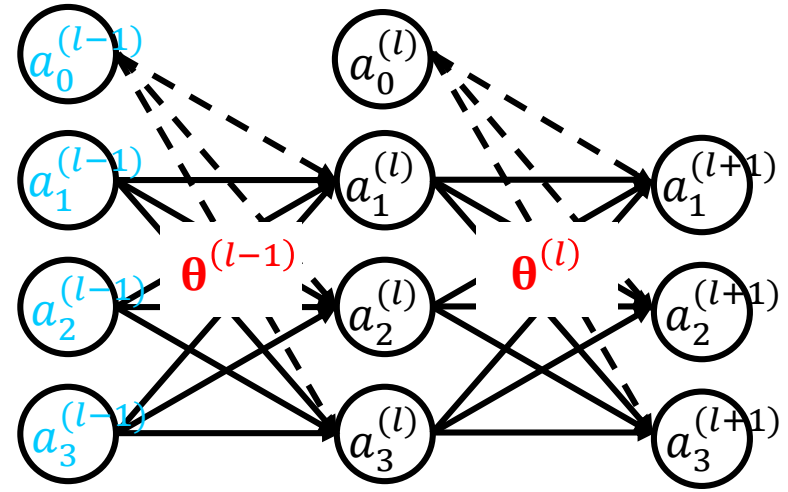


Computation of error terms $\delta^{(l)}$

- For l th and $(l + 1)$ th layers we have
$$\begin{cases} \mathbf{z}^{(l)} = \boldsymbol{\theta}^{(l-1)} \mathbf{a}^{(l-1)} \\ \mathbf{a}^{(l)} = g(\mathbf{z}^{(l)}) \\ \mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)} \\ \mathbf{a}^{(l+1)} = g(\mathbf{z}^{(l+1)}) \end{cases}$$
- Therefore, the cost function depends on $\mathbf{z}^{(l)}$ through later layers $\mathbf{z}^{(l+1)}$

$$\mathbf{z}^{(l)} \rightarrow \mathbf{a}^{(l)} \rightarrow \mathbf{z}^{(l+1)} \rightarrow \mathbf{a}^{(l+1)} \rightarrow J(\boldsymbol{\theta})$$

$$\mathbf{z}^{(l)} = \boldsymbol{\theta}^{(l-1)} \mathbf{a}^{(l-1)} \quad \mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)}$$

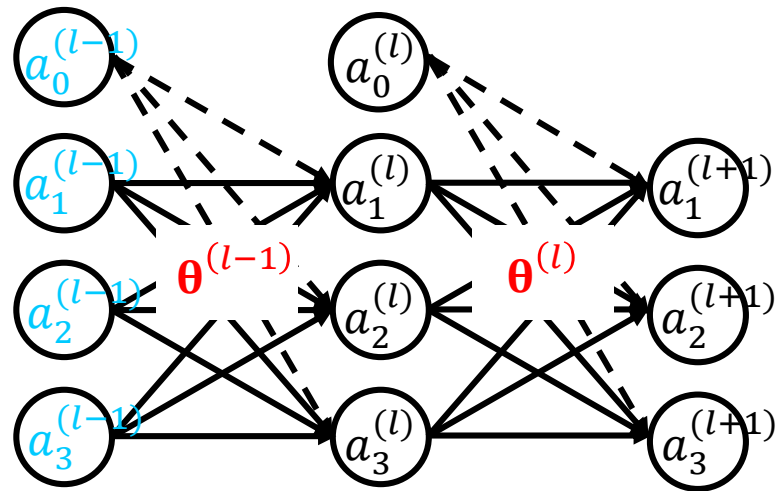


Computation of error terms $\delta^{(l)}$

$$\mathbf{z}^{(l)} = \boldsymbol{\theta}^{(l-1)} \mathbf{a}^{(l-1)} \quad \mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)}$$

- The variable $z_k^{(l)}$ has an impact on each k unit in layer $l + 1$, thus chain rule applies as follows

$$\begin{aligned} \delta_k^{(l)} &= \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l)}} \\ &= \sum_{k=0} \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} \end{aligned}$$



Computation of error terms $\delta^{(l)}$

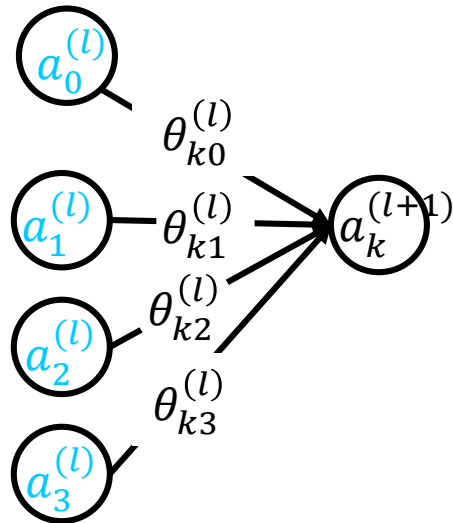
$$\mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)}$$

- Recall

$$\begin{aligned} z_k^{(l+1)} &= [\boldsymbol{\theta}^{(l)}]_{k*} \mathbf{a}^{(l)} \\ &= \theta_{k0}^{(l)} a_0^{(l)} + \theta_{k1}^{(l)} a_1^{(l)} + \theta_{k2}^{(l)} a_2^{(l)} + \theta_{k3}^{(l)} a_3^{(l)} \\ &= \sum_{j=0}^3 \theta_{kj}^{(l)} a_j^{(l)} \end{aligned}$$

- Now, we compute

$$\begin{aligned} \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} &= \frac{\partial \sum_{j=0}^3 \theta_{kj}^{(l)} a_j^{(l)}}{\partial z_i^{(l)}} \\ &= \theta_{ki}^{(l)} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \end{aligned}$$



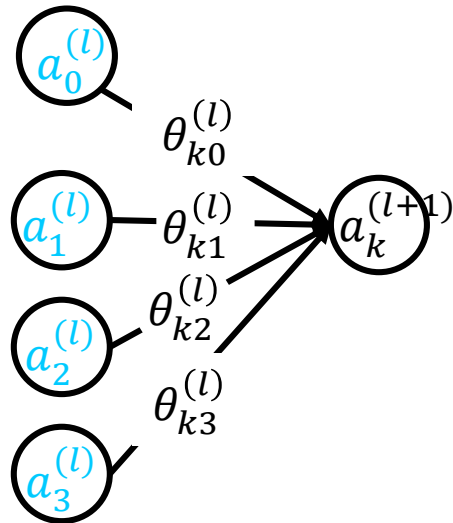
$[\mathbf{A}]_{k*}$ denotes the k th row of the matrix $\mathbf{A}$

Computation of error terms $\delta^{(l)}$

$$\mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)}$$

Finally,

$$\begin{aligned}\delta_k^{(l)} &= \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l)}} = \sum_{k=0} \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} \frac{\partial g(z_i^{(l)})}{\partial z_i^{(l)}}\end{aligned}$$



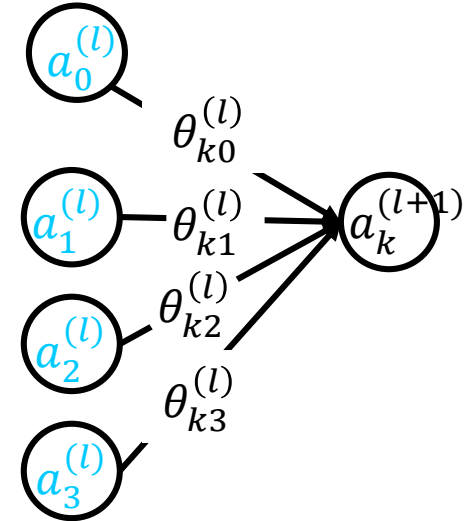
Computation of error terms $\delta^{(l)}$

$$\mathbf{z}^{(l+1)} = \boldsymbol{\theta}^{(l)} \mathbf{a}^{(l)}$$

Finally,

$$\begin{aligned}\delta_k^{(l)} &= \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l)}} = \sum_{k=0} \frac{\partial J(\boldsymbol{\theta})}{\partial z_k^{(l+1)}} \frac{\partial z_k^{(l+1)}}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} \frac{\partial a_i^{(l)}}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} \frac{\partial g(z_i^{(l)})}{\partial z_i^{(l)}} \\ &= \sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} g'(z_i^{(l)})\end{aligned}$$

- Here, $\sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)}$ is the i th element of $(\boldsymbol{\theta}^{(3)})^T \boldsymbol{\delta}^{(4)}$



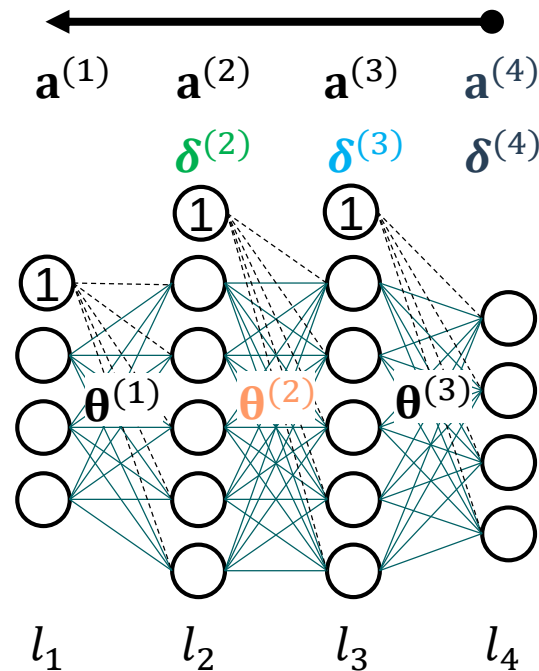
Error terms in compact form

Layer 4 (output):

$$\delta^{(4)} = (\mathbf{a}^{(4)} - \mathbf{y}) \in R^{4 \times 1}$$

Layer 3:

$$\delta^{(3)} = (\boldsymbol{\theta}^{(3)})^T \delta^{(4)} \odot g'(\mathbf{z}^{(3)}) \in R^{5 \times 1}$$



Remark: Derivative of the sigmoid function

- We assumed the sigmoid activation function

$$g(x) = \frac{1}{1 + e^{-x}}$$

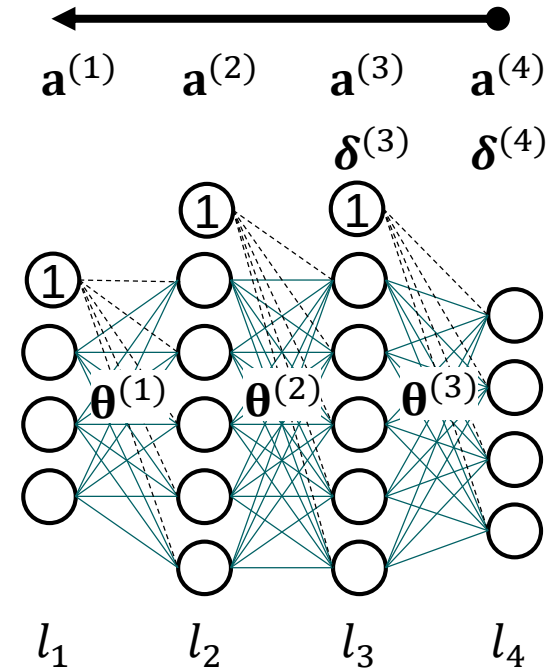
- The derivative of activation function at the input value $z_i^{(l)}$ has a form

$$\begin{aligned} g'(z_i^{(l)}) &= g(z_i^{(l)}) (1 - g(z_i^{(l)})) \\ &= a_i^{(l)} (1 - a_i^{(l)}) \end{aligned}$$

Or in vector form:

$$g'(\mathbf{z}^{(l)}) = \mathbf{a}^{(l)} \odot (1 - \mathbf{a}^{(l)})$$

$\odot$ denotes element-wise multiplication



Error terms in compact form

Layer 4 (output):

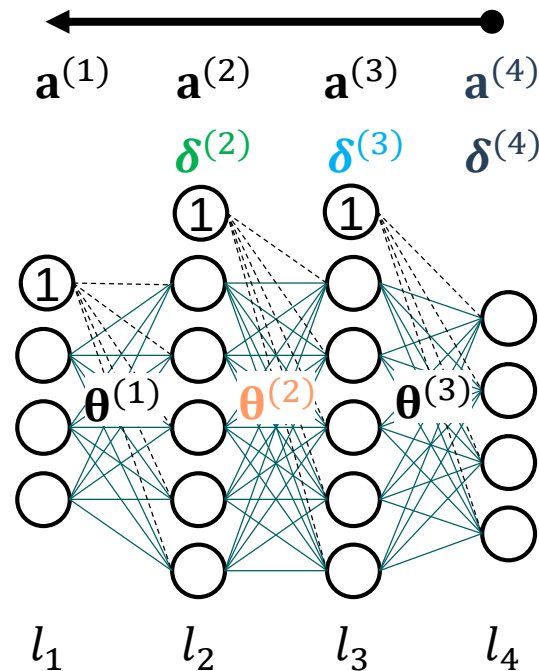
$$\delta^{(4)} = (\mathbf{a}^{(4)} - \mathbf{y}) \in R^{4 \times 1}$$

Layer 3:

$$\delta^{(3)} = (\boldsymbol{\theta}^{(3)})^T \delta^{(4)} \odot g'(\mathbf{z}^{(3)}) \in R^{5 \times 1}$$

Layer 2:

$$\delta^{(2)} = (\boldsymbol{\theta}^{(2)})^T \delta^{(3)} \odot g'(\mathbf{z}^{(2)}) \in R^{5 \times 1}$$



Note: There is no error term $\delta^{(1)}$ for the input layer

Computation of gradient $\frac{\partial J(\theta)}{\partial \theta^{(l)}}$ (with regularization)

- Now, we use **error terms** to compute the gradient for each of the layers!

Layer 3:

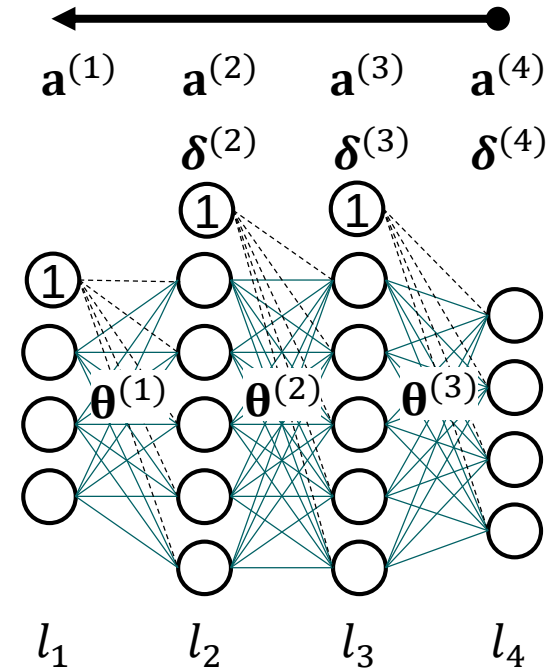
$$\frac{\partial J(\theta)}{\partial \theta^{(3)}} = \frac{\partial J(\theta)}{\partial \mathbf{z}^{(4)}} \frac{\partial \mathbf{z}^{(4)}}{\partial \theta^{(3)}} = \frac{1}{m} \delta^{(4)} (\mathbf{a}^{(3)})^T + \frac{\lambda}{m} \theta^{(3)} \in R^{4 \times 5}$$

Layer 2:

$$\frac{\partial J(\theta)}{\partial \theta^{(2)}} = \delta^{(3)} (\mathbf{a}^{(2)})^T + \frac{\lambda}{m} \theta^{(2)} \in R^{5 \times 5}$$

Layer 1:

$$\frac{\partial J(\theta)}{\partial \theta^{(1)}} = \delta^{(2)} (\mathbf{a}^{(1)})^T + \frac{\lambda}{m} \theta^{(1)} \in R^{5 \times 3}$$



$$\mathbf{a}^{(1)} = \mathbf{x}$$

Computation of gradient $\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}_0^{(l)}}$ for bias parameters $\boldsymbol{\theta}_0^{(l)}$

Layer 3:

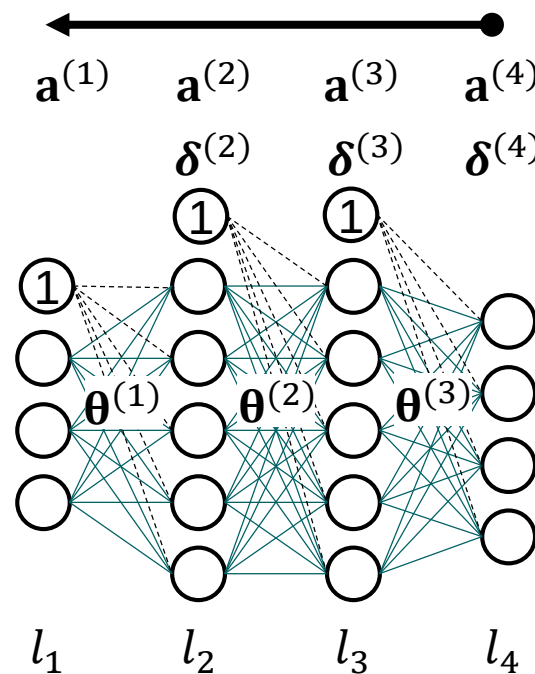
$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0^{(3)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(4)}} \frac{\partial \mathbf{z}^{(4)}}{\partial \theta_0^{(3)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(4)}} \cdot 1 = \boldsymbol{\delta}^{(4)} \in R^{4 \times 1}$$

Layer 2:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0^{(2)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(3)}} \cdot 1 = \boldsymbol{\delta}^{(3)} \in R^{5 \times 1}$$

Layer 1:

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0^{(1)}} = \frac{\partial J(\boldsymbol{\theta})}{\partial \mathbf{z}^{(2)}} \cdot 1 = \boldsymbol{\delta}^{(2)} \in R^{5 \times 1}$$





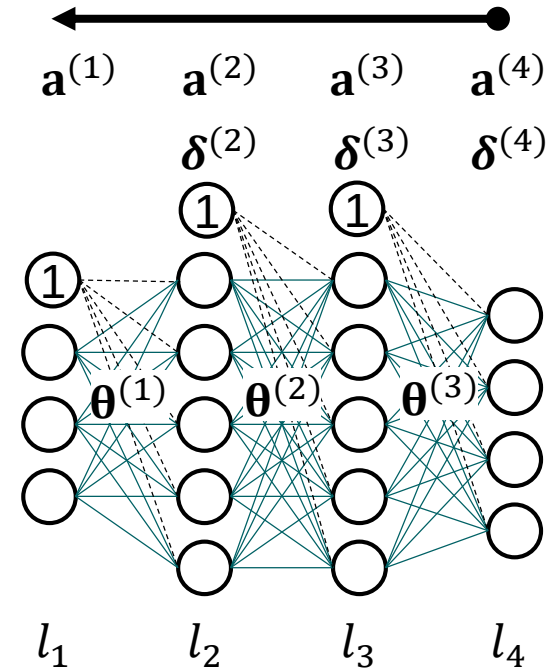
Parameter update with gradient descent computation

Computation of gradient $\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}}$

- Generally, for $l = 1 \sim 3$ we have

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}} = \boldsymbol{\delta}^{(l+1)} (\mathbf{a}^{(l)})^T$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0^{(l)}} = \boldsymbol{\delta}^{(l+1)}$$



Update of parameters

- For each layer, we update the parameters by using the computed gradients $\frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}}$ as follows

$$\begin{cases} \boldsymbol{\theta}^{(l)} \leftarrow \boldsymbol{\theta}^{(l)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}^{(l)}} \\ \theta_0^{(l)} \leftarrow \theta_0^{(l)} - \alpha \frac{\partial J(\boldsymbol{\theta})}{\partial \theta_0^{(l)}} \end{cases}$$

- Note:** Don't forget that bias parameters: $\theta_0^{(1)}$, $\theta_0^{(2)}$ and $\theta_0^{(3)}$ are only updated during the backward computation!



Summary

Parameter initialization

- For gradient descent and advanced optimization, we need to initiate the values of θ
- Consider gradient descent: start from some value and continue going downhill
- Two options:
 - $\theta_{ij}^{(l)} = 0, (\forall i, j, l)$
 - Logistic regression: OK
 - NN is not OK
 - $\theta_{ij}^{(l)} = \text{RANDOM}, (\forall i, j, l)$

Backpropagation algorithm procedure

Set $\frac{\partial J(\theta)}{\partial \theta^{(l)}} = 0$ ($\forall l$)

for $i = 1$ to m // computation for all training examples $(x^{(i)}, y^{(i)})_{i=1 \sim m}$

Set $\mathbf{a}^{(1)} = \mathbf{x}^{(i)}$

Compute $\mathbf{a}^{(l)}$ for $l = 2, 3, \dots, L$ as forward propagation step

Initialize output error $\delta^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}^{(i)}$

Compute errors $\delta^{(L-1)}, \delta^{(L-2)}, \dots, \delta^{(2)}$ as

$$\delta^{(L-1)} = (\theta^{(L-1)})^T \delta^{(L)} \odot g'(\mathbf{z}^{(L-1)})$$

Compute gradients $\frac{\partial J(\theta)}{\partial \theta^{(l)}} = \delta^{(l+1)} (\mathbf{a}^{(l)})^T$

End

Update parameters $\theta^{(l)} \leftarrow \theta^{(l)} - \alpha \frac{1}{m} \sum_{i=1}^m \frac{\partial J(\theta)}{\partial \theta^{(l)}}$

Final remarks

- The best number of hidden units depend on:
 - number of inputs and outputs
 - number of training examples
 - the amount of noise in the targets
 - the complexity of the function to be learned
 - the activation function

Too few hidden units:

=> high training and generalization error, due to underfitting and high statistical bias

Too many hidden units:

=> low training error but high generalization error, due to overfitting and high variance

Thank you!

